Qualcomm Technologies, Inc.

IPQ50xx SoC Software

User Guide

80-16052-17 Rev. H

July 2, 2021

For additional information or to submit technical questions, go to https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2020-2021 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
A	July 2020	Initial release
B	September 2020	Updated Section 5.3 Tiny-NOR single image generation
C	December 2020	Updated: ■ Section 3.4 eMMC flash layout ■ Section 3.5 NOR + eMMC flash layout Added Section 11.1.7 Sysupgrade support with eMMC GPT partition update
D	January 2021	Updated Chapter 12, SBL UART customization
E	March 2021	Added Section 6.8, Q6 boot arguments
F	April 2021	Updated Section 11.3, Board recovery using USB emergency download mode
G	May 2021	Updated Chapter 10, Debug interfaces
H	July 2021	Updated Section 6.2, Serial bus interface

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Contents

Revision history 2

1 IPQ50xx architecture and software overview 8

1.1 IPQ50xx CPU architecture and system features 8

1.1.1 Power management 9

1.1.2 CPU governor and A53 CPU frequency 9

1.1.3 Reference software platform (QSDK) 9

2 Boot process 11

2.1 Primary boot loader 11

2.2 Secondary boot loader 12

2.3 Qualcomm secure extension environment 12

2.4 Applications boot loader 12

3 Memory layout 13

3.1 NOR flash layout 13

3.2 NAND flash layout 14

3.2.1 NAND flash blocks and pages 14

3.2.2 Bad block management 15

3.3 NOR + NAND flash layout 16

3.4 eMMC flash layout 17

3.5 NOR + eMMC flash layout 18

3.6 DDR layout 19

3.7 OCIMEM layout 20

4 Flash customization 21

4.1 Customize the NOR and NAND partition table 21

4.1.1 Add or modify a NOR or NAND partition entry 21

4.1.2 Example NOR or NAND partition entry 21

4.2 Customize the eMMC partition table 22

4.2.1 Add or modify an eMMC partition entry 23

4.2.2 Example eMMC partition entry 23

4.3 Customize eMMC flash size 23

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4 Add a SPI NOR device 24
4.4.1 Example: SPI NOR flash, 64 K block size, and 32 MBytes density 24
4.4.2 Example: SPI NOR flash, 4 K block size, and 2 MBytes density 25
4.5 Generate more partition tables 26
5 Build a single image 27
5.1 Prepare the input folder for a single image 27
5.2 Pack the single image using pack.py 29
5.3 Tiny-NOR single image generation 30
6 Linux configurations 33
6.1 Linux device tree 33
6.1.1 Linux pin control drivers 33
6.1.2 Configure pin as GPIO using DTS 33
6.1.3 Register devices using DTS 34
6.2 Serial bus interface 34
6.3 Network configuration 35
6.3.1 GMAC interfaces 35
6.3.2 Enable/disable net_device features during runtime using ethtool 35
6.3.3 net_device MAC address 36
6.4 Ethernet switch and PHYs 36
6.5 Register for panic or reboot notifications 36
6.6 Register for Wi-Fi restart notifications 37
6.7 Disable unnecessary flash types and peripherals in Linux 37
6.8 Q6 boot arguments 40
7 U-Boot configurations 41
7.1 Remove peripherals 41
7.2 Ethernet support 41
7.2.1 GEPHY configuration 42
7.2.2 UNIPHY configuration 42
7.2.3 MP0x U-Boot support for QCA8081, QCA8033 PHYs and S17C switch 42
7.3 Tiny U-Boot 43
7.4 Disable unnecessary flash types and peripherals in U-Boot 44
7.5 Machine ID 45
8 Combined UBI image for NAND flash 46
9 Include QCN90xx firmware image 48
10 Debug interfaces 49
10.1 Set up Lauterbach 50

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2 TRACE32 scripts 50
10.3 Debug using JTAG 50
10.3.1 Disable CONFIG_PERF_EVENTS when using JTAG debug 51
10.3.2 Launch T32 52
10.3.3 Set up kernel debug 53
10.3.4 Debug modules 54
10.3.5 Multicore debug 56
11 Flash image update procedure and non-bootable board recovery 57
11.1 Options to flash the software image 57
11.1.1 Image selection 57
11.1.2 Transfer the image using TFTP 58
11.1.3 Transfer the image using USB (host mode) 58
11.1.4 Reference kit 58
11.1.5 Flash the image and perform a hard reboot 61
11.1.6 Sysupgrade support 61
11.1.7 Sysupgrade support with eMMC GPT partition update 62
11.2 Recover using JTAG 62
11.3 Board recovery using USB emergency download mode 66
11.4 Flash Wi-Fi firmware-only images using TFTP 72
11.4.1 NAND boot and NOR + NAND boot 72
11.4.2 NOR boot 72
11.4.3 eMMC boot and NOR + eMMC boot 73
11.5 Flash Bluetooth firmware-only images using TFTP 73
11.5.1 NAND boot and NOR + NAND boot 74
11.5.2 NOR boot 74
11.5.3 eMMC boot and NOR + eMMC boot 75
12 SBL UART customization 76

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Tables

Table 3-1: 32 MByte partitions....13
Table 3-2: 16 MByte partitions....13
Table 3-3: OCIMEM layout....20
Table 5-1: Tiny-NOR partition layout....31
Table 6-1: Linux DTS values/supported configurations....35
Table 7-1: U-Boot-2016 supported configurations for QTI reference boards....42
Table 11-1: IPQ50xx AP.MP03 interface details....60

Figures

Figure 2-1: Boot process workflow..... 11

Figure 11-1: AP.MP03 board..... 59

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1 IPQ50xx architecture and software overview

This document describes the functionalities of IPQ50xx software along with the boot process, flash memory specifications, and Linux pin control drivers of the IPQ50xx chipsets with Qualcomm® Internet Processor (IPQ) technology.

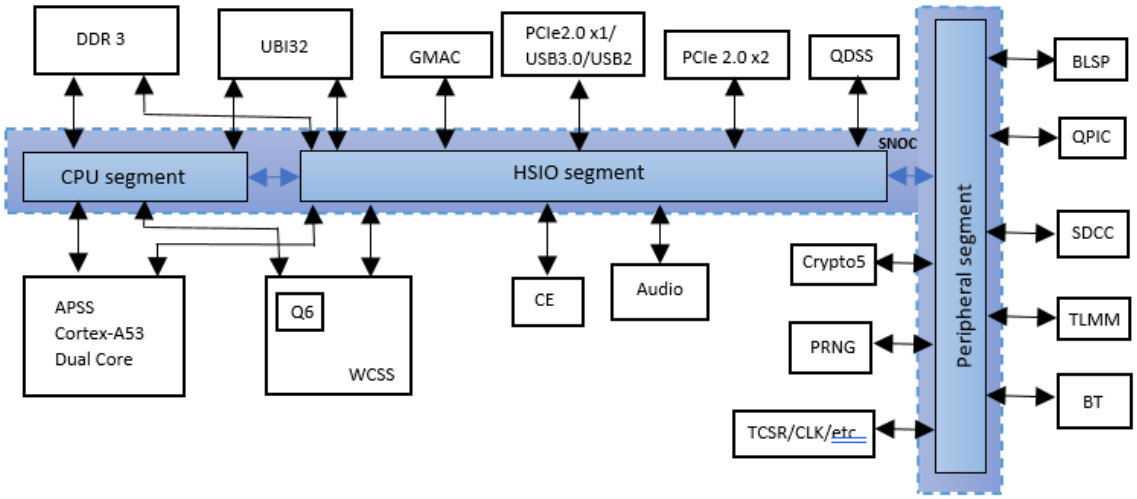
FCC NOTICE:

The IPQ50xx software kit is designed to allow:

- Product developers to evaluate electronic components, circuitry, or software associated with the kit to determine whether to incorporate such items in a finished product and
- Software developers to write software applications for use with the final product. This kit is not a finished product. When assembled, do not resell or otherwise market it without all required FCC equipment authorizations. Operation is subject to the condition that this product does not cause harmful interference to licensed radio stations and that this product accepts harmful interference. The operator of the kit must operate under the authority of an FCC license holder or must secure an experimental authorization under part 5 of the FCC rules. That is, unless the assembled kit is designed to operate under part 15, part 18, or part 95 of the FCC rules.

1.1 IPQ50xx CPU architecture and system features

The following diagram shows the IPQ50xx CPU architecture and system features:



CPU subsystem	Principal processing load
Dual Quad Cortex-A53 application processors	SMP Linux OS, networking applications, for example, Wi-Fi SON, upper layers of networking protocol stacks, control protocol processing such as DHCP, DNS, ICMP.
NSS	Exception cases for common networking protocols, Wi-Fi <> wired forwarding/bridging, complex protocol offload such as LAG, GRE, MAP-T.
Hexagon QDSP6-Wi-Fi	Responsible for Wi-Fi connectivity
BTSS	Responsible for Bluetooth connectivity

1.1.1 Power management

The following power management techniques are used in the IPQ50xx chipset:

A53 CPU frequency scaling	1 GHz CPU frequency is supported for all variants.
NSS CPU frequency scaling	1 GHz NSS CPU frequency is supported for all variants.
DDR and NoC	■ DDR3 runs at 800 MHz ■ DDR4 runs at 266.67 MHz ■ Peripheral segment runs at 100 MHz ■ Frequency scaling is not supported for DDR and SNoC

1.1.2 CPU governor and A53 CPU frequency

The performance governor is supported and this is the default governor.

1.1.3 Reference software platform (QSDK)

The Qualcomm Software Development Kit (QSDK) provides all reference software required to boot SMP Linux on a Qualcomm Technologies, Inc. (QTI) reference platform, initialize, and operate all network interfaces, and develop customer-specific applications. QSDK for IPQ50xx has the following components:

- Linux kernel
- Secondary Boot Loader (SBL)
- Qualcomm® Trusted Execution Environment software
- U-Boot: High-level boot loader for Linux
- OpenWrt: Embedded Linux distribution reference software and framework for customizing and building a networking application from existing and value-add components
- Board configuration file for the reference designs supplied by QTI
- Application profiles that tailor the OpenWrt networking software to suit various application classes
- SoC drivers
- Qualcomm® Wi-Fi Solutions proprietary driver package

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The IPQ50xx chipset includes several cryptographic engines useful for accelerating cryptographic operations. QSDK includes a choice of cryptographic APIs that help software work with the cryptographic engines:

- Standard Linux CryptoAPI
- OpenBSD Cryptographic Framework (OCF)

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2 Boot process

This chapter describes the boot operation workflow and the different boot loader components. On the IPQ50xx chipsets, Cortex-A53 core 0 is released from reset when the power rails are within specification and starts running the Primary Bootloader (PBL). Cortex-A53 core 0 initiates a chain-of-trust sequence of events that use file and image authentication to ensure that the intended and trusted firmware and software components are loaded. The chain-of-trust continues until the software is loaded into DDR.

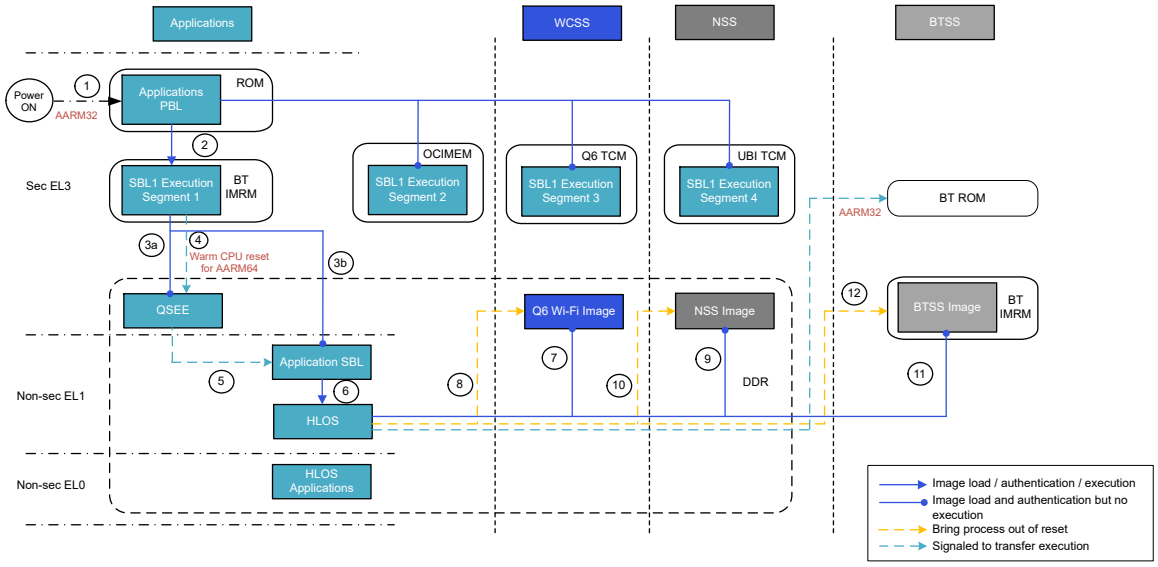


Figure 2-1 Boot process workflow

2.1 Primary boot loader

The primary boot loader (PBL) is burnt into the boot ROM of the Cortex-A53 processor at the factory. Its functions are:

- Hardware initialization
- Boot device detection: eMMC/NOR/NAND (serial)
- Load SBL from the first block of the detected boot device into the system: WCSS IMEM, UBI TCM, OCIMEM, and BT RAM
- Authenticate SBL image
- Transfer control to SBL image

2.2 Secondary boot loader

The secondary boot loader (SBL) performs boot-specific initialization before transferring control to the application boot loader. It runs on a single core of the Cortex-A53 processor. The SBL functions are:

- Initialize DDR memory using DDR parameters read from flash.
- Load the QSEE image; can read the QSEE image from NOR/NAND/eMMC to PCDDR3.
- Load APPSBL; can read the SBL image from NOR/NAND/eMMC to PCDDR3.
- Hand over control to QSEE.

2.3 Qualcomm secure extension environment

The Qualcomm secure extension environment (QSEE) software runs in the privilege mode of TrustZone (TZ).

QSEE functions include:

- Perform security configuration of the SoC
- Brings the secondary cores out of reset
- Performs context dump collection for crash dumps
- Offers secure run-time services to HLOS via the SCM interface
- Secure boot
- Access control
  - The xPUs control access and are enabled by default. For debug purposes, access control alone can be disabled by flashing devcfg_noac.mbn in the DEVCFG partition.

2.4 Applications boot loader

The primary functions of the U-Boot applications boot loader (APPSBL) are:

- Load the kernel into PCDDR memory and execute
- Pass boot arguments to the kernel

U-Boot also enables reflashing of the kernel and the file system. It can utilize these interfaces by default:

SPI-NOR flash	Store and retrieve kernel, file system, and so on
NAND flash	Store and retrieve kernel, file system, and so on
eMMC flash	Store and retrieve kernel, file system, and so on
Ethernet	Upgrade the SBL, QSEE, U-Boot, kernel, and file system via TFTP

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3 Memory layout

The IPQ50xx flash memory organization is useful for loading images into flash memory and booting the device using the image from flash memory.

3.1 NOR flash layout

Table 3-1 32 MByte partitions

Partition	Size (KBytes)	Comments
SBL1	192	Secondary boot loader
MIBIB	64	Partition table
QSEE	512	TZ/ATF
DEVCFG	64	Dynamic xPU configuration
CDT	64	Platform ID and DDR configuration
APPSBLENV	64	U-Boot ENV variables
APPSBL	640	U-Boot
ART	448	Wi-Fi calibration data
HLOS	5120	Kernel and DTB
Rootfs	20480	File system
WIFIFW	4096	Wi-Fi firmware partition
BTFW	256	Bluetooth firmware

Table 3-2 16 MByte partitions

Partition	Size (KBytes)	Comments
BOOTLDR1	192	SBL + CDT
BOOTLDR2	384	QSEE + DEVCFG + U-BOOT + MIBIB
APPSBLENV	64	U-Boot ENV variables
ART	64	Wi-Fi calibration data
HLOS	1600	Kernel + DTB
Rootfs	6848	Root file system
WIFIFW	3392	Wi-Fi firmware

3.2 NAND flash layout

Partition	Size (KBytes)	Comments
SBL1	512	Secondary boot loader
MIBIB	512	Partition table
BOOTCONFIG	256	Failsafe partition information
BOOTCONFIG1	256	Failsafe partition information
QSEE	1024	TZ/ATF
QSEE_1	1024	Alternate partition
DEVCFG	256	Dynamic xPU configuration
DEVCFG_1	256	Alternate partition
CDT	256	Platform ID and DDR configuration
CDT_1	256	Alternate partition
APPSBLENV	512	U-Boot ENV variables
APPSBL	1280	U-Boot
APPSBL_1	1280	Alternate partition
ART	1024	Wi-Fi calibration data
TRAINING	512	QSPI serial training partition
rootfs	59392	Ubinize Kernel + DTB + file system + Wi-Fi firmware + BT firmware
rootfs_1	59392	Alternate partition

When a system upgrade is complete, the alternate partitions become the active partitions and vice versa. For each system upgrade, alternate and active partitions get switched.

3.2.1 NAND flash blocks and pages

The NAND flash structure contains blocks and pages:

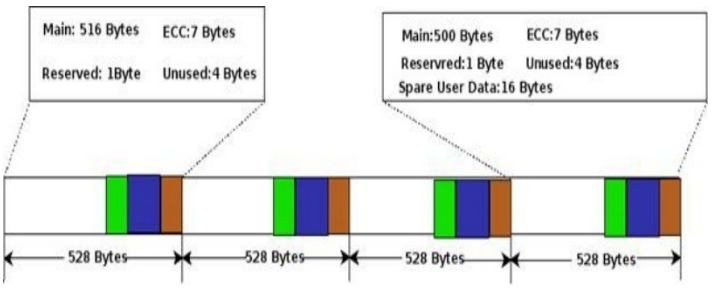
Blocks	Smallest erasable unit of a NAND flash; broken up into units called pages
Pages	Smallest programmable unit of a NAND flash; broken up into units called main data and spare data

The NAND controller (which is part of the QTI chipset) has a built-in ECC engine for ECC detection and correction on pages that are read. It calculates the ECC for a page to write and stores it in the ECC area of the page spare region. The IPQ50xx chipset uses the 4/8-bit BCH ECC algorithm. In the page layout, the flash layout is uniform across all SBL components up to the Linux kernel.

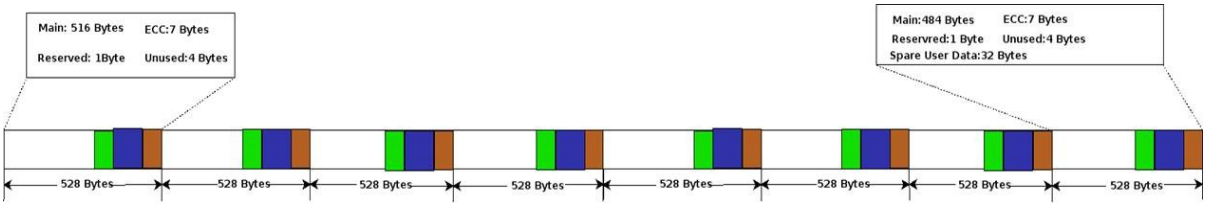
For 4-bit BCH ECC

2K page:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

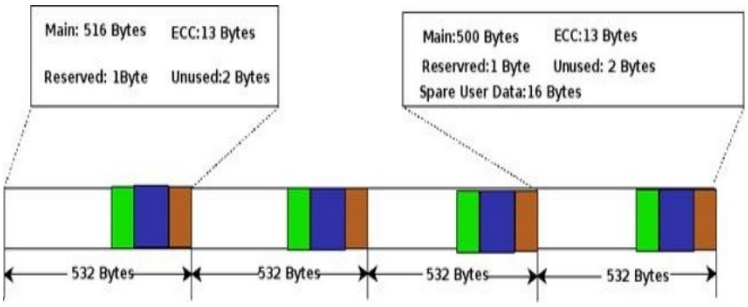


4K page



For 8-bit BCH ECC

2K page



4K page



3.2.2 Bad block management

Bad blocks contain one or more invalid bits whose reliability is not guaranteed. Bad blocks may be present when the device is shipped or may develop during the lifetime of the device.

The IPQ50xx chipset uses the skip block method scheme to manage bad blocks. The algorithm starts by reading the spare area of the first page of each block and checks if the first or the second byte is not 0xff. Based on this, bad block table is created. This process occurs at every boot. An image is sequentially programmed page-by-page into the target flash device. When the target address corresponds to an invalid (that is, bad) block address, that block is skipped and the data is stored in the next good block. Since the invalid block is skipped, the data in the spare array indicating the presence of the invalid block is kept. Thus, the system can build a table of invalid blocks addresses by reading the spare area of all the blocks at boot time. Because the invalid block exists, user data should not exceed the size of good blocks.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A graphical representation of bad block scheme is as follows:

NAND Block 0	Image in Block 0
1	Img-Block 1
2	Img-Block 2
Block 3 is bad	Skip
4	Img-Block 3
5	Img-Block 4
6	Img-Block 5
Block 7 is bad	Skip
8	Img-Block 6
9	Img-Block 7
Block 10 is bad	Skip
11	Img-Block 8
12	Img-Block 9
.....	.....
n-4	Img-Block 10
n-4	Img-Block 11
n-2	Img-Block 12
Block n-1	Skip
n	Img-Block n-4
n+1	Img-Block n-3
n+2	Img-Block n-2
n+3	Img-Block n-1
n+4	Img-Block n

Image
Img-Block 1
Img-Block 2
Img-Block 3
Img-Block 4
Img-Block 5
Img-Block 6
Img-Block 7
Img-Block 8
Img-Block 9
Img-Block 10
Img-Block 11
Img-Block 12
.....
Img-Block n-4
Img-Block n-3
Img-Block n-2
Img-Block n-1
Img-Block n

3.3 NOR + NAND flash layout

NOR			NAND		
Partition	**Size (KBytes)**	**Comments**	**Partition**	**Size (KBytes)**	**Comments**
SBL1	192	Secondary boot loader	Rootfs	59392	Ubinizekernel + DTB + File system + Wi-Fi FW + BT FW
MIBIB	64	Partition table	rootfs_1	59392	Alternate partition
BOOTCONFIG	64	Failsafe partition information			
BOOTCONFIG1	64	Failsafe partition information			

NOR			NAND		
Partition	Size (KBytes)	Comments	Partition	Size (KBytes)	Comments
QSEE	512	TZ			
QSEE_1	512	TZ alternate partition			
DEVCFG	64	Dynamic xPU configuration			
DEVCFG_1	64	Alternate partition dynamic xPU configuration			
CDT	64	Platform ID and DDR configuration			
CDT_1	64	Alternate partition platform ID and DDR configuration			
APPSBLENV	64	U-Boot ENV variables			
APPSBL	640	U-Boot			
APPSBL_1	640	Alternate partition U-Boot			
ART	448	Wi-Fi calibration data			

3.4 eMMC flash layout

Partition	Size (KBytes)	Comments
GPT	17	Partition table
SBL1	256	Secondary boot loader
BOOTCONFIG	128	Failsafe partition information
BOOTCONFIG1	128	Failsafe partition information
QSEE	640	TrustZone (TZ)
QSEE_1	640	Alternate partition
DEVCFG	128	Dynamic xPU configuration
DEVCFG_1	128	Alternate partition
CDT	128	Platform ID and DDR configuration
CDT_1	128	Alternate partition
APPSBLENV	256	U-Boot ENV Variables
APPSBL	640	U-Boot
APPSBL_1	640	Alternate partition
ART	512	Wi-Fi Calibration data
HLOS	6144	Kernel+ DTB
HLOS_1	6144	Alternate partition
rootfs	61440	File system

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Partition	Size (KBytes)	Comments
rootfs_1	61440	Alternate partition
WIFIFW	6144	Wi-Fi firmware partition
WIFIFW_1	6144	Alternate Wi-Fi firmware partition
BTFW	256	Bluetooth Firmware
BTFW_1	256	Alternate BTFW
rootfs_data	131072	Filesystem data
BackupGPT	16.5	Backup partition table

When a system upgrade is complete, the alternate partitions become the active partitions and vice versa. For each system upgrade, alternate and active partitions are switched.

3.5 NOR + eMMC flash layout

NOR			eMMC		
Partition	Size (KBytes)	Comments	Partition	Size (KBytes)	Comments
SBL1	192	Secondary boot loader	GPT	17	Partition table
MIBIB	64	Partition table	HLOS	6144	Kernel+ DTB
BOOTCONFIG	64	Failsafe partition information	HLOS_1	6144	Alternate partition
BOOTCONFIG 1	64	Failsafe partition information	rootfs	61440	File system
QSEE	640	TZ/ATF	rootfs_1	61440	Alternate partition
QSEE_1	640	Alternate partition TZ	WIFIFW	6144	Wi-Fi FW partition
DEVCFG	64	Dynamic xPU configuration	WIFIFW_1	6144	Alternate partition
			BTFW	256	Bluetooth Firmware
			BTFW_1	256	Alternate Bluetooth FW
DEVCFG_1	64	Alternate partition dynamic xPU configuration	rootfs_data	131072	Filesystem data
CDT	64	Platform ID and DDR configuration	BackupGPT	16.5	Backup partition table
CDT_1	64	Alternate partition platform ID and DDR configuration			
APPSBLENV	64	U-Boot ENV variables			
APPSBL	640	U-Boot			

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

NOR			eMMC		
Partition	Size (KBytes)	Comments	Partition	Size (KBytes)	Comments
APPSBL_1	640	Alternate partition U-Boot			
ART	448	Wi-Fi calibration data			

3.6 DDR layout

The DDR layout is defined in **arch/arm64/boot/dts/qcom/qcom-ipq5018-memory.dtsi**. The memory is reserved for other subsystems in the reserved-memory node.

512 MB DDR layout

Partition	Start address	Size (MBytes)
NSS UBI32	0x40000000	16
Linux HLOS	0x41000000	Remaining memory
U-Boot	0x4A600000	4
SBL	0x4AA00000	1
QSEE	0x4AC00000	4
SMEM	0x4AB00000	1
WLAN AREA	0x4B000000	27
WLAN Q6 ETR	0x4CB00000	1
QCN90xx Q6	0x4CD00000	40
BT	0x07000000	0.34

256 MB DDR layout

Partition	Start address	Size (MBytes)
NSS UBI32	0x40000000	
Linux HLOS	0x41000000	
U-Boot	0x4A600000	
SBL	0x4AA00000	
QSEE	0x4AC00000	
SMEM	0x4AB00000	
WLAN AREA	0x4B000000	
WLAN Q6 ETR	0x4D800000	
BT	0x07000000	
QCN90xx Q6	0x4CD00000	

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.7 OCIMEM layout

The IPQ50xx chipset supports 32 KB of On-Chip Internal Memory (OCIMEM). This memory contents are retained only after the warm reset and not during the cold reset. The OCIMEM range is from 0x8600000 to 0x8608000.

Table 3-3 OCIMEM layout

Memory	Component	Start address	Size
SHARED IMEM	SBL	0x08600000	0XC8 (200 Bytes)
	USB	0x086000C8	0XC8
	CDT	0x08600190	0XC8
	Reserved	0x08600590	0XC8
	HLOS	0x08600658	0X64
	Reserved for OEM	0x086006BC	0x64 (100 bytes)
	TZ	0x08600720	0X338

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4 Flash customization

4.1 Customize the NOR and NAND partition table

Edit the XML file for the corresponding flash to customize the NOR and NAND partition table. The XML files for the flash configurations are available in QSDK; the git project is `system/tools/meta`.

The meta directory contains all scripts and configuration files. The IPQ50xx-specific partition files are available at **apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/flash_partition/**

IPQ50xx partition file	XML file
NOR	nor-partition.xml
NAND	nand-partition.xml
NOR + NAND	norplusnand-partition.xml
NOR + eMMC	norplusemmc-partition.xml

4.1.1 Add or modify a NOR or NAND partition entry

1. Copy an existing partition entry.
2. Add an entry or modify the existing entry based on these rules:
   - In the partition XML file, specify the size in the **size_kb** option. If **size_kb** is used, the last attribute should be 0xFF.
   - If the **size_kb** option is 0xFFFFFFFF, it is taken as grow partition. That is, all remaining space is used for that partition. Only the last partition can have a grow partition size.
   - Do not reorder the partition 0:SBL1, as it should always be the first entry.

4.1.2 Example NOR or NAND partition entry

To add a partition named **CONFIG** with a size of 4 MBytes and a pad size of 1 MB in the NAND:

1. Edit the master copy of the file **apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/flash_partition/nand-partition.xml.**

   This master copy is copied by the **python_update_common_info.py script** to the working directory, **./common/build/ipq(_64)** used for assembling the component images that are packed into the final single image.

   ```
   . . .
   <partition>
   ```

```
<name length="16" type="string">0:CONFIG</name>
<size_kb length="4">4096</size_kb>
<pad_kb length="4">1024</pad_kb>
<which_flash>0</which_flash>
<attr>0xFF</attr>
<attr>0xFF</attr>
<attr>0x00</attr>
<attr>0xFF</attr>
</partition>
<partition>
. . .
```

2. Regenerate the system partition by running this command from the meta directory:

   ```
   python prepareSingleImage.py --arch ipq5018 --fltype nor,nand,emmc –
   genpart -in <images path>
   ```

   Where:

--arch	Specifies the platform type; that is, ipq5018
--fltype	Specifies the flash type (provide multiple flash types with a comma)
--in	Output directory where all individual images are copied for single image packing

   For example, these commands regenerate the system partition for the IPQ5018 partition files for a 32-bit image:

Partition file	Command to regenerate system partition	Output file
NOR board	python prepareSingleImage.py --arch ipq5018 --fltype nor --genpart --in /images/ipq	common/build/ipq/nor- system-partition- ipq5018.bin
NAND board	python prepareSingleImage.py --arch ipq5018 --fltype nand --genpart --in /images/ipq	common/build/ipq/nand-system-partition-ipq5018.bin
NOR+NAND board	python prepareSingleImage.py --arch ipq5018 --fltype norplusnand --genpart --in /images/ipq	common/build/ipq/norplusnand-system-partition-ipq5018.bin
NOR + eMMC board	python prepareSingleImage.py --arch ipq5018 --fltype norplusemmc --genpart --in / images/ipq	common/build/ipq/norplusemmc-system-partition-ipq5018.bin

4.2 Customize the eMMC partition table

To customize the eMMC partition table, edit the file:

apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/flash_partition/emmc-partition.xml

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.2.1 Add or modify an eMMC partition entry

1. Copy an existing partition entry.
2. Add or modify the entry based on these rules:
   - Specify the partition name and size (KBytes) in the partition XML.
   - Type specifies the GUID of the specific partition. Do not change the type for SBL, TZ, or APPSBL.
   - Generate a new GUID for the newly added partition. Specify the file to be flashed; to create an empty partition, leave the filename field empty.

4.2.2 Example eMMC partition entry

To add a configuration partition named 0:CONFIG with a size of 50 KBytes in eMMC:

1. Edit the file apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/flash_partition/emmc-partition.xml based on:

```
<partition label="0:CONFIG" size_in_kb="50" type="0CCE190E-
C1E9-4CED-9E1D-590A75C5205C" bootable="false" readonly="false"
filename="config.mbn"/>
```

2. Run the command from the ./common/build/ipq or ipq_x64 directory depending on whether 32-bit or 64-bit images are being generated:

Partition file	Command to regenerate system partition	Output file
eMMC board	python prepareSingleImage.py --arch ipq5018 --fltype emmc --genpart --in / images/ipq	gpt_main0.bin gpt_backup0.bin

4.3 Customize eMMC flash size

To customize the eMMC flash size, edit the ./apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/config.xml as follows:

1. Modify the total_block (block count) per the data sheet:

```
<data type="EMMC_PARAMETER">
<total_block>7634944</total_block>
    <partition_mbn>gpt_main0.bin</partition_mbn>
                <partition_mbn_backup>gpt_backup0.bin</
partition_mbn_backup>
</data>
```

2. Run this command from the**./common/build/ipq or ipq_x64** directory depending on whether 32-bit or 64-bit images.

Partition file	Command to regenerate GPT	Output file
eMMC board	python prepareSingleImage.py --arch ipq5018 --fltype emmc --genpart --in / images/ipq	gpt_main0.bin gpt_backup0.bin

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4 Add a SPI NOR device

To add support for a new SPI NOR device:

1. Edit the partition XML files (see Section 4.1) for the partition in: apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/flash_partition/
2. Update these entries in apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq5018/config.xml:
   - pagesize
   - pages_per_block
   - total_blocks
   - Flash block size (in KBytes)
   - Flash density (in MBytes)
3. Generate the system partition.
4. Regenerate the system partition using these commands from the meta directory:

Partition File	Command to regenerate system partition	Output File
NOR board	python prepareSingleImage.py --arch ipq5018 --fltype nor --genpart --in /images/ipq	</images/ipq/nor-system-partition-ipq5018.bin>
NOR+NAND board	python prepareSingleImage.py --arch ipq5018 --fltype norplusnand --genpart --in /images/ipq	</images/ipq/norplusnand-system-partition-ipq5018.bin>

NOTE Non-JEDEC flash devices are not supported for fallback.

4.4.1 Example: SPI NOR flash, 64 K block size, and 32 MBytes density

- Edit the partition XML file as follows:

```
<partition>
    <name length="16" type="string">0:MIBIB</name>
   <size_kb length="4">128</size_kb>
   <pad_kb length="4">0</pad_kb>
   <which_flash>0</which_flash>
   <attr>0xFF</attr>
   <!-- Specify flash block size in KB -->
   <attr>64</attr>
   <!-- Specify flash density in MB -->
   <attr>32</attr>
   <attr>0xFF</attr>
   <img_name type="string">nor-user-partition-ipq5018.bin</img_name>
</partition>
```

- Edit the config.xml file as follows:

```
<data type="NOR_PARAMETER">
<page_size>256</page_size>
<pages_per_block>256</pages_per_block>
```

```
<total_block>512</total_block>
</data>
```

4.4.2 Example: SPI NOR flash, 4 K block size, and 2 MBytes density

- Edit the partition XML file as follows:

```
<partition>
    <name length="16" type="string">0:MIBIB</name>
   <size_kb length="4">128</size_kb>
   <pad_kb length="4">0</pad_kb>
   <which_flash>0</which_flash>
   <attr>0xFF</attr>
   <!-- Specify flash block size in KB -->
   <attr>4</attr>
   <!-- Specify flash density in MB -->
   <attr>2</attr>
   <attr>0xFF</attr>
   <img_name type="string">nor-user-partition-ipq5018.bin</img_name>
 </partition>
```

- Edit the config.xml as follows:

```
<data type="NOR_PARAMETER">
<page_size>256</page_size>
<pages_per_block>16</pages_per_block>
<total_block>512</total_block>
</data>
```

The fail-safe upgrade feature is enabled by default. It uses BOOTCONFIG and the partitions ending with _1. If this fail-safe boot feature is not required, these partitions may be removed:

- BOOTCONFIG
- BOOTCONFIG1
- QSEE_1
- DEVCFG_1
- CDT_1
- APPSBL_1
- HLOS_1
- rootfs_1
- WIFIFW_1
- BTFW_1

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.5 Generate more partition tables

The scripts use the default partition XMLs to generate partition tables. Use these commands to generate a table from more XMLs.

Script	Location
nand_mbn_generator.py, ptoool.py	meta-tools/scripts
msp.py	
partition_tool.exe	boot_images/QcomPkg/IPQ5018Pkg/Tools/

For NAND

```
python nand_mbn_generator.py <partition.xml> <user partition file>
partition_tool -s <nand_page_size> -p <nand_pages_per_block> -b
<nand_total_block_count> -u <user partition file> -o <output_system_partition>
```

For NOR

```
python nand_mbn_generator.py <partition.xml> <user partition file>
partition_tool -s <nor_page_size> -p <nor_pages_per_block> -b
<nor_total_block_count> -c 1 -u <user partition file> -o
<output_system_partition>
```

For NOR + NAND

```
python nand_mbn_generator.py <partition.xml> <user partition file>
partition_tool -s <nor_page_size> -p <nor_pages_per_block> -b
<nor_total_block_count> -x <nand_page_size> -y <nand_pages_per_block> -z
<nand_total_block> -c 1 -u <user partition file> -o <output_system_partition>
```

For eMMC

```
python ptool.py -x <emmc_partition.xml>
python msp.py -r rawprogram0.xml  -p patch0.xml -d <total blocks> -n
```

For NOR + eMMC

Generate the NOR partition:

```
python nand_mbn_generator.py <partition.xml> <user partition file>
partition_tool -s <nor_page_size> -p <nor_pages_per_block> -b
<nor_total_block_count> -c 1 -u <user partition file> -o
<output_system_partition>
```

Generate the eMMC partition

```
python ptool.py -x <emmc_partition.xml>
python msp.py -r rawprogram0.xml  -p patch0.xml -d <total blocks> -n
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5 Build a single image

QSDK includes single image script packages. The git project is system/tools/meta. To build a single image, prepare the input folder with all the required components needed for a single image using prepareSingleImage.py and generate the final single image file using **pack.py**.

5.1 Prepare the input folder for a single image

A single image file consists of many different component images. The first step in creating a single image is to prepare the input folder for pack.py. The prepareSingleImage.py options include:

SingleImage.py option	Argument	Description
--arch	<*chip name*>	The chip family: ■ ipq5018: IPQ5018 32-bit ■ ipq5018_64: IPQ5018 64-bit
--fltype	<*flash list*>	Indicates flashes to generate images for; specify multiple flashes using comma-separated format. ■ *nand* – NAND flash ■ *nor* – NOR flash ■ *tiny-nor* – Tiny-NOR flash ■ *emmc* – eMMC flash ■ *norplusnand* – NOR + NAND flash
--in	<*path*>	Input directory contains: ■ Tools such as partition_tool and bootconfig_tool ■ Generated binaries such as MIBIBs and CDTs
--inImage	<*path*>	Path to the folder that contains all images (the same path where **preparesingleimage.py** generated the images).
--bootimg	<*path*>	If specified, boot images available at <*path*> are copied to the path specified with **–in** option.
--tzimg	<*path*>	If specified, TZ images available at <*path*> are copied to the path specified with **–in** option.
--rpmimg	<*path*>	If specified, RPM images available at <*path*> are copied to the path specified with **–in** option.
--nhssimg	<*path*>	If specified, NHSS images available at <*path*> are copied to the path specified with **–in** option.
--gencdt		Indicates whether to generate CDT binaries; the argument does not take any value specified CDT binary and is generated in the path specified with **–in** option.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

SingleImage.py option	Argument	Description
--genpart		Indicates whether to generate partition tables; the argument does not take any value; if specified partition tables are generated in the path specified with **–in** option.
--genbootconf		Indicates whether to generate bootconfig binaries; the argument does not take any value; if specified, bootconfig binaries are generated in the path specified with **–in** option.
--genmbn		Indicates whether to convert U-Boot elf to **u-boot.mbn**; argument does not take any value; if specified, **u-boot.mbn** is not generated in the path specified with **–in** option.
--genbootldr		Indicates whether to generate bootldr images for Tiny-NOR single-image; the argument does not take any value; if specified, bootldr1 and bootldr2 images are generated in the path specified with **–in** option.
--help		Prints help message.

Example: 32 bit NOR, NAND, eMMC, generate CDT, partition table, and boot config for IPQ50xx

```
$ python prepareSingleImage.py --arch ipq5018 --fltype nor,nand,emmc --gencdt
--genpart --genbootconf --in ./ipq/
```

Example: 64 bit NOR, partition table for IPQ50xx

```
$ python prepareSingleImage.py --arch ipq5018_64 --fltype nor --genpart --
in ./ipq/
```

Example: 32 bit NAND, partition table for IPQ50xx

```
$ python prepareSingleImage.py --arch ipq5018 --fltype nand --gencdt --
in ./ipq/
```

Example: 32 bit NAND (4K Page size, 256 KiB block size), partition table for IPQ5018

1. Modify partition configurations. Update the size_kb attribute for rootfs and rootfs_1 partitions in: ipq/ipq5018/flash_partition/nand-partition.xml.

```
<partition>
     <name length="16" type="string">rootfs</name>
     <size_kb length="4">59392</size_kb>
     <pad_kb length="2">256</pad_kb>
     <which_flash length="2">0</which_flash>
     <attr>0xFF</attr>
     <attr>0xFF</attr>
     <attr>0x00</attr>
     <attr>0xFF</attr>
     <img_name type="string" mode="32" image_type="hlos">openwrt-ipq-
ipq50xx-ubi-root.img
     </img_name>
     <img_name type="string" mode="64" image_type="hlos">openwrt-ipq-
ipq50xx_64-ubi-root.img
     </img_name>
```

```
</partition>
<partition>
     <name length="16" type="string">rootfs_1</name>
     <size_kb length="4">59392</size_kb>
     <pad_kb length="2">256</pad_kb>
     <which_flash length="2">0</which_flash>
     <attr>0xFF</attr>
     <attr>0xFF</attr>
     <attr>0x00</attr>
     <attr>0xFF</attr>
          </partition>
```

2. Modify the page_size attribute for NAND_PARAMETER in: ipq/ipq5018/config.xml

```
<data type="NAND_PARAMETER">
         <page_size>4096</page_size>
         <pages_per_block>64</pages_per_block>
         <total_block>4096</total_block>
          </data>
```

3. Generate the system partition.

```
python prepareSingleImage.py --arch ipq5018 --fltype nand,norplusnand --
genpart --in ./ipq/
```

5.2 Pack the single image using pack.py

1. Pack the single image using pack.py, which is available at: qsdk/qca/src/u-boot-2016/tools/pack.py
2. Use pack.py to generate images for NOR, NAND, eMMC, NOR+NAND, and NOR+eMMC. The pack script options include:

Packscript option	Argument	Description	
--arch	<*chipname*>	The chip family: ■ *ipq5018*:IPQ5018 32-bit ■ *ipq5018_64*: IPQ501864-bit	
--fltype	<*flashlist*>	Indicates flashes to generate the images for. Specify multiple flashes using comma-separated format.	
		nand	NAND flash
		nor	NOR flash
		emmc	eMMC flash
		norplusnand	NOR + NAND flash
		norplusemmc	NOR + eMMC flash
--srcPath	<*path*>	Path to the metatools folder.	
--inImage	<*path*>	Path to the folder that contains all images (the same path where **preparesingleimage.py** generated the images).	
--outImage	<*path*>	Path to generate single images.	

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Example: 32 bit, all flashes for IPQ5018

```
python /date/qsdk/qca/src/u-boot-2016/tools/pack.py
            --arch ipq5018
            --fltype nand,norplusnand,emmc,norplusemmc,nor
            --srcPath /date/qsdk/../meta-tools
            --inImage /date/ipq
            --outImage /date/ipq/bin
```

Example: 32 bit eMMC for IPQ5018

```
python /date/qsdk/qca/src/u-boot-2016/tools/pack.py
            --arch ipq5018
            --fltype emmc
            --srcPath /date/qsdk/../meta-tools
            --inImage /date/ipq
            --outImage /date/ipq/bin
```

Example: 64 bit, all flashes for IPQ5018

```
python /date/qsdk/qca/src/u-boot-2016/tools/pack.py
            --arch ipq5018_64
            --fltype nand,norplusnand,emmc,norplusemmc,nor
            --srcPath /date/qsdk/../meta-tools
            --inImage /date/ipq
            --outImage /date/ipq/bin
```

Example: 64 bit, NOR for IPQ5018

```
python /date/qsdk/qca/src/u-boot-2016/tools/pack.py
            --arch ipq5018_64
            --fltype nor
            --srcPath /date/qsdk/../meta-tools
            --inImage /date/ipq
            --outImage /date/ipq/bin
```

5.3 Tiny-NOR single image generation

The 16M default (Tiny-NOR) and debug (Tiny-NOR-debug) builds use compressed boot-loader images, and kernel and U-Boot images are built with low memory profile configurations.

The Tiny-NOR flash is supported with 16 MB profile and the config source file is available at: */qsdk/qca/configs/qsdk/ipq_16M.config*

The new flash layouts for Tiny-NOR and Tiny-NOR-debug with 16 MB support are included for IPQ5018 with changes to the existing partitions. BOOTLDR1 and BOOTLDR2 partitions are added; BOOTLDR1 is the combined image of [SBL + CDT] and BOOTLDR2 is the combined image of [MIBIB + QSEE + DEVCFG + U-BOOT].

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The partition layout XML files are available at *meta-tools/ipq5018/ flash_partition/tiny-nor-partition.xml* and *meta-tools/ipq5018/ flash_partition/tiny-nor-debug-partition.xml.*

Table 5-1 Tiny-NOR partition layout

Partition	Comments
BOOTLDR1	SBL + CDT
BOOTLDR2	MIBIB + QSEE + DEVCFG + APPSBL
APPSBLENV	Uboot ENV variables
ART	Wi-Fi calibration data
HLOS	Kernel + DTB
rootfs	Root file system
WIFI FW	Wi-Fi firmware
BT FW	Bluetooth firmware

Generate the Tiny-NOR single image

1. Build the Kernel and U-Boot tiny version images with ipq_16M.config.
2. Build the SBL tiny version image.
3. Copy the required tiny version images to the ipq/ directory.
   - SBL1_tiny_nor.elf
   - tiny_mon.elf
   - tiny_qsee.elf
   - devcfg.elf
   - openwrt-ipq5018_tiny-u-boot.elf (or) openwrt-ipq5018_tiny_debug-u-boot.elf
4. Generate the BOOTLDR1 image.

   Include the –genbootldr option in preprareSingleImage.py command. This option invokes the *gen_bootldr1.sh* located at scripts/ipq5018 script to generate BOOTLDR1 images specific to each support board using the respective CDT binary.

   ```
   python ./ipq/prepareSingleImage.py --arch ipq5018 --fltype <tiny-nor/tiny-nor-debug>
   --genbootldr --in ./ipq/
   ```

5. Generate the BOOTLDR2 image.

   ```
   */gen_bootldr.sh <script path> <image path> <[tiny-nor]/tiny-nor-debug>
   ```

6. Pack the Tiny-NOR single image using pack.py.

   ```
   python ./pack.py --arch ipq5018 --fltype <tiny-nor/tiny-nor-debug> --
   srcPath ./ipq
   --inImage ./ipq --outImage ./ipq/bin
   ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Flash the BOOTLDR images

- Partition: BOOTLDR1

```
tftpb bootldr1_<board_name>_LM256.mbn
Flash 0:BOOTLDR1
```

  Example: `bootldr1_AP-MP02.1_128M16_DDR3_LM256.mbn`

- Partition: BOOTLDR2

```
tftpb bootldr2_nor.mbn
Flash 0:BOOTLDR2
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6 Linux configurations

6.1 Linux device tree

The Linux operating system (OS) uses a device tree (DTS) in:

- 32-bit device tree: arch\arm\boot\dts\
- 64-bit device tree: arch\arm64\boot\dts\qcom

6.1.1 Linux pin control drivers

The Linux pinctrl implementation provides a framework to configure pins with drive strength, pull up, and so on, as well as supporting multiplexing of pins for different use cases. The clients/consumers of pins interact with the framework through a combination of device tree bindings and interface APIs.

- Kernel documentation on pinctrl is available at: **Documentation/pinctrl.txt**
- Target-specific documentation for pinctrl is available at **Documentation/devicetree/bindings/pinctrl/qcom,ipq5018-pinctrl.txt**

  Target-specific documentation includes details on the available pins, supported functions, and usage examples.

6.1.2 Configure pin as GPIO using DTS

By default, all pins are in the GPIO mode. To configure a pin for an alternate functionality, specify the function name in DTS using the function property. If no alternative functionality is specified, the default function of GPIO remains.

Example: Configure pins 7 and 9 as GPIO

```
leds_pins: led_pinmux {
                  mux {
                       pins = "gpio7", "gpio9";
                       drive-strength = <2>;
                       bias-none;
                       output-high;
                       };
                     };
gpio-leds {
            compatible = "gpio-leds";
```

```
                pinctrl-0 = <&leds_pins>;
};
```

6.1.3 Register devices using DTS

Use DTS to find and register devices in the system.

- Kernel documentation on the device tree is available at:

  Documentation/devicetree/usage-model.txt

- Target-specific documentation is available at:

  Documentation/devicetree/bindings

For better maintenance, the 32-bit DTS includes the DTS files from the 64-bit DTS directory. Make any changes in DTS for both 32-bit and 64-bit builds in: **arch\arm64\boot\dts\qcom**

The IPQ5018 device tree files include:

- qcom-ipq5018-coresight.dtsi
- qcom-ipq5018-db-mp02.1.dts
- qcom-ipq5018-db-mp03.1-c2.dts
- qcom-ipq5018-db-mp03.1.dts
- qcom-ipq5018-db-mp03.3-c2.dts
- qcom-ipq5018-db-mp03.3.dts
- qcom-ipq5018.dtsi
- qcom-ipq5018-memory.dtsi
- qcom-ipq5018-mhi.dtsi
- qcom-ipq5018-mp02.1.dts
- qcom-ipq5018-mp03.1-c2.dts
- qcom-ipq5018-mp03.1.dts
- qcom-ipq5018-mp03.3-c2.dts
- qcom-ipq5018-mp03.3.dts
- qcom-ipq5018-thermal.dtsi

S No	RDP	DTS
1	MP02.1	qcom-ipq5018-mp02.1.dts
2	MP03.1, MP03.1-c2 (eMMC boot)	qcom-ipq5018-mp03.1.dts, qcom-ipq5018-mp03.1-c2.dts
3	MP03.3, MP03.3-c2 (eMMC boot)	qcom-ipq5018-mp03.3.dts, qcom-ipq5018-mp03.3-c2.dts

6.2 Serial bus interface

The IPQ5018 chipset uses BLSP for the serial bus interface. There are 2 UART instances and 3 QUP (SPI or I2C) instances. UART and QUP supports BAM mode transfers. The channel allocation is listed here.

Interface	Tx Channel	Rx Channel	interrupt request (IRQ) number
BLSP1_BLSP_UART0 @ 0x078AF000	0	1	107
BLSP1_BLSP_UART1 @ 0x078B0000	2	3	108
BLSP1_BLSP_QUP0 @ 0x078B5000	4	5	95
BLSP1_BLSP_QUP1 @ 0x078B6000	6	7	96
BLSP1_BLSP_QUP2 @ 0x078B7000	8	9	97

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.3 Network configuration

The IPQ50xx chipset uses the qca-nss-dp driver for physical interfaces. By default, AP.MP03.1 reference platform enables two 1 Gbps ports. It assigns eth0 which uses the IPQ50xx internal 1 GB PHY as the WAN interface, and eth1 which is connected to S17c switch as LAN interface. The AP.MP03.3 reference platform has the 2.5 GB PHY connected to eth1. eth1 is added to the bridge 'br-lan' to be used in bridged mode. The default configuration is:

```
config interface 'lan'
option ifname 'eth1'
option force_link '1'
option type 'bridge' option proto 'static'
option ipaddr '192.168.1.1'
option netmask '255.255.255.0'
option ip6assign '60'
config interface 'wan'
option ifname 'eth0'
option proto 'dhcp'
```

6.3.1 GMAC interfaces

There are two GMAC interfaces – one is connected to internal GE PHY with GMII connection, and the other is SGMII/SGMII plus connection with external switch/PHY. The interface modes are defined in the device tree as `switch_mac_mode`.

The modes supported by Linux Device Tree for current reference boards are listed here.

Table 6-1 Linux DTS values/supported configurations

	Port 1: GMAC0	Port 2: GMAC1
PORT_WRAPPER_SGMII_PLUS		X
PORT_WRAPPER_SGMII		X

6.3.2 Enable/disable net_device features during runtime using ethtool

Use **ethtool** to enable and disable certain net_device features during runtime. By default, all net_device features are enabled. The commands to enable/disable these features at runtime are:

```
ethtool –K ethX tx off/on   /*  Disable/enable tx checksum  */
ethtool –K ethX rx off/on   /*  Disable/enable rx checksum  */
ethtool –K ethX tso off/on  /* Disable/enable TCP segmentation */
ethtool –K ethX sg off/on   /* Disable/enable scatter gather */
ethtool –K ethX gro off/on  /* Disable/enable generic receive offload */
ethtool –K ethX gso off/on  /* Disable/enable generic segmentation offload */
ethtool –S ethX             /* Read statistics */
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.3.3 net_device MAC address

The ART partition stores all interface MAC addresses. U-Boot reads the ART partition and updates the DTS parameter local-mac-address; then qca-nss-dp parses it and uses it to configure the net_device MAC address. If U-Boot has not configured a MAC address, qca-nss-dp assigns the net_device a random MAC address.

6.4 Ethernet switch and PHYs

There is one internal GE PHY with 10M/100M/1000M speed. AP-MP02 supports external QCA803x, AP-MP03.1 supports external QCA8337, and AP-MP03.3 supports external QCA808x. This is defined in ess-switch node in each board DTS.

Example: AP-MP02.1: Internal GE PHY+ one QCA803x

```
ess-instance {
    num_devices = <0x1>;
    ess-switch@0x39c00000 {
        switch_mac_mode = <0xf>;
        qcom,port_phyinfo {
            port@0 {
                port_id = <1>;
                phy_address = <7>;
                mdiobus = <&mdio0>;
            };
            port@1 {
                port_id = <2>;
                phy_address = <0x4>;
                mdiobus = <&mdio1>;
            };
        };
    };
};
```

6.5 Register for panic or reboot notifications

Drivers that require panic notifications or reboot notifications can use kernel APIs to register a callback for these events.

Panic notifications

Use the API **atomic_notifier_chain_register** to register the notifier handler to the **panic_notifier_list**. See the file **drivers/watchdog/qcom-wdt.c** for an example.

Restart notifications

Use the API **register_reboot_notifier** to register the notifier handler. See the file **drivers/soc/qcom/restart_reason.c** for an example.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.6 Register for Wi-Fi restart notifications

On a Wi-Fi firmware crash, the host attempts to collect crash information and then restart the Q6 processor. The restart module can provide notifications here to registered kernel mode components.

Use the API **subsys_notif_register_notifier** to register the notifier handler, with the subsystem name **qcom_q6v5_wcss**. See the file **drivers/soc/qcom/glink_ssr.c** for an example. These events are supported:

Event	Description
SUBSYS_BEFORE_SHUTDOWN	Sent before Q6 shutdown process begins.
SUBSYS_AFTER_SHUTDOWN	Sent after Q6 is successfully shut down.
SUBSYS_BEFORE_POWERUP	Sent before Q6 powerup sequence is attempted.
SUBSYS_AFTER_POWERUP	Sent after Q6 is successfully powered up.
SUBSYS_RAMDUMP_NOTIFICATION	Sent when RAM dump collection starts in the event of a Q6 crash.
SUBSYS_POWERUP_FAILURE	Sent if Q6 powerup sequence fails.

6.7 Disable unnecessary flash types and peripherals in Linux

Flash

By default, all flash types are initialized in the Kernel. To disable the unnecessary flash types:

Flash type	To disable the flash type if it is not needed:
NOR	1. Change the status of the node spi_0 from ok to disabled in qsdk/qca/src/linux-4.4/arch/arm64/boot/dts/qcom/qcom-ipq5018-mp03.1.dts. 2. Remove these lines in qsdk/target/linux/ipq/config-4.4: ▫ CONFIG_MTD_SPI_NOR=y
NAND	1. Change the status of the node nand from ok to disabled in qsdk/qca/src/linux-4.4/arch/arm64/boot/dts/qcom/qcom- ipq5018-mp03.1.dts 2. Remove these lines in qsdk/target/linux/ipq/config-4.4: ■ CONFIG_MTD_NAND=y ■ CONFIG_MTD_NAND_ECC=y ■ CONFIG_MTD_NAND_SERIAL=y ■ CONFIG_PAGE_SCOPE_MULTI_PAGE_READ=y ■ CONFIG_MTD_NAND_QCOM=y ■ CONFIG_MTD_UBI=y ■ CONFIG_MTD_UBI_BEB_LIMIT=20 ■ CONFIG_MTD_UBI_BLOCK=y ■ CONFIG_MTD_UBI_FASTMAP is not set

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Flash type	To disable the flash type if it is not needed:
	■ CONFIG_MTD_UBI_GLUEBI=y ■ CONFIG_MTD_UBI_WL_THRESHOLD=4096
eMMC	1. Change the status of the node sdhc_1 from ok to disabled in qsdk/qca/src/linux-4.4/arch/arm64/boot/dts/qcom/qcom- ipq5018-mp03.1-c2.dts. 2. Remove these lines in qsdk/target/linux/ipq/config-4.4: ▫ CONFIG_MMC=y ▫ CONFIG_MMC_ARMMMCI=y ▫ CONFIG_MMC_BLOCK=y ▫ CONFIG_MMC_BLOCK_MINORS=32 ▫ CONFIG_MMC_QCOM_DML=y ▫ CONFIG_MMC_QCOM_TUNING=y ▫ CONFIG_MMC_SDHCI=y ▫ CONFIG_MMC_SDHCI_MSM=y ▫ # CONFIG_MMC_SDHCI_OF_ARASAN is not set ▫ # CONFIG_MMC_SDHCI_PCI is not set ▫ CONFIG_MMC_SDHCI_PLTFM=y ▫ # CONFIG_MMC_TIFM_SD is not set

Peripherals

By default, these peripherals are enabled. To disable the unneeded peripherals on the board:

Peripheral	To disable the peripheral if it is not needed:
USB	1. Remove these lines in qsdk/target/linux/ipq/Makefile: ▫ From FEATURES: – usbgadget ▫ From DEFAULT_PACKAGES: – kmod-usb2 – kmod-usb3 – kmod-usb-f-qdss 2. Remove these lines in target/linux/ipq/ipq50xx/target.mk and target/linux/ipq/ipq50xx_64/target.mk: ▫ From DEFAULT_PACKAGES: – kmod-usb-dwc3-qcom 3. Remove these lines from qsdk/target/linux/ipq/profiles/qsdk.mk: ▫ kmod-usb-storage, kmod-usb-uas and “USB_ETHERNET:= kmod-usb-net-rtl8152 kmod-usb-net” ▫ From PACKAGES: – “$(USB_ETHERNET)” references ▫ From USB_DIAG: – kmod-usb-f-diag

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Peripheral	To disable the peripheral if it is not needed:
	4. Remove this line from qsdk/target/linux/ipq/config-4.4: ▫ CONFIG_USB_SUPPORT=y
I2C	Change the statuses of these nodes from ok to disabled in qsdk/qca/src/linux-4.4/arch/arm64/boot/dts/qcom/qcom-ipq5018-mp03.1.dts: ■ i2c_0
PWM	To disable PWM, do the following in file qsdk/target/linux/ipq/ipq50xx/config-default and qsdk/target/linux/ipq/ipq50xx_64/config-defaultt: ■ Remove the following line: ▫ CONFIG_PWM_IPQ5018=y ■ Add the following line: ▫ # CONFIG_PWM_IPQ5018 is not set
CoreSight	In qsdk/target/linux/ipq/ipq50xx/config-default and qsdk/target/linux/ipq/ipq50xx_64/config-default: 1. Remove these lines: ▫ CONFIG_SPS=y ▫ CONFIG_SPS_SUPPORT_NDP_BAM=y ▫ CONFIG_CORESIGHT=y ▫ CONFIG_CORESIGHT_CSR=y ▫ CONFIG_CORESIGHT_CTI=y ▫ CONFIG_CORESIGHT_EVENT=y ▫ CONFIG_CORESIGHT_HWEVENT=y ▫ CONFIG_CORESIGHT_LINKS_AND_SINKS=y ▫ CONFIG_CORESIGHT_LINK_AND_SINK_TMC=y ▫ CONFIG_CORESIGHT_QCOM_REPLICATOR=y ▫ # CONFIG_CORESIGHT_QPDI is not set ▫ # CONFIG_CORESIGHT_SINK_ETBV10 is not set ▫ CONFIG_CORESIGHT_SINK_TPIU=y ▫ CONFIG_CORESIGHT_SOURCE_ETM3X=y ▫ CONFIG_CORESIGHT_SOURCE_ETM4X=y ▫ # CONFIG_CORESIGHT_REMOTE_ETM is not set ▫ CONFIG_CORESIGHT_STM=y ▫ CONFIG_CORESIGHT_TPDA=y ▫ CONFIG_CORESIGHT_TPDM=y ▫ # CONFIG_CORESIGHT_TPDM_DEFAULT_ENABLE is not set ▫ # CONFIG_CORESIGHT_SOURCE_DUMMY is not set 2. Add this line: ▫ # CONFIG_CORESIGHT is not set
Bluetooth	Remove kmod-bt_tty from DEFAULT_PACKAGES in ■ 32 bit: "target/linux/ipq/ipq50xx/target.mk" ■ 64 bit: "target/linux/ipq/ipq50xx_64/target.mk"

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.8 Q6 boot arguments

The IPQ50xx Q6 firmware does not have a flexible way to configure runtime information based on the reference design platform (RDP). There is no equivalent for the device tree in the Q6 firmware. This can be mitigated by using the Linux device tree. The configuration information can be specified in the Linux device tree for the RDP, and the Q6 image loader passes this information to the Q6 firmware. This is similar to the boot loaders passing boot arguments to Linux.

The boot arguments for Q6 firmware are defined in the Type, Length, Value (TLV) format. The allowed types and their formats are listed here.

Description	Type (Byte)	Length (Byte)	Arg1 (Byte)	Arg2 (Byte)	Arg 3 (Byte)	Arg4 (Byte)
PCIe configuration information for QCN61xx in Slot 0	1	4	3	PERST GPIO Number	0	0
PCIe configuration information for QCN61xx in Slot 1	2	4	2	PERST GPIO Number	0	0

The boot arguments must be defined in the RDP device tree; the `boot-args` definition is available in the board-specific dts file under `qcom_q6v5_wcss` node.

```
boot-args = <Type Length Arg1 Arg2 Arg3 Arg4>, <Type Length Arg1 Arg2 Arg3
Arg4>;
```

The boot arguments are updated in the dts file depending on the configuration.

Examples

- To configure GPIO 0x10 for QCN61xx in Slot 0 and GPIO 0x12 for QCN61xx in Slot 1, update:

  ```
  boot-args = <0x1 0x4 0x3 0x10 0x0 0x0>, <0x2 0x4 0x2 0x12 0x0 0x0>;
  ```

- To configure GPIO 0x12 for QCN61xx connected in Slot 0, update:

  ```
  boot-args = <0x1 0x4 0x3 0x12 0x0 0x0>;
  ```

- To configure GPIO 0x5 for QCN61xx connected in Slot 1, update:

  ```
  boot-args = <0x2 0x4 0x2 0x5 0x0 0x0>;
  ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7 U-Boot configurations

The newest U-Boot versions support a sub-set of the Linux Device Tree Structure. The U-boot-2016 device tree files are available at *qsdk/qca/src/u-boot-2016/arch/arm/dts/ipq5018-*.dts*, where *ipq5018-soc.dtsi* defines the defaults for all reference boards.

7.1 Remove peripherals

Users can remove these peripherals:

spi	If the target does not support SPI flash in U-Boot
nand	If the target does not support NAND flash in U-Boot
xhci	If the target does not support USB interface in U-Boot
i2c	If the target does not support I2C interface in U-Boot
pcie	If the target does not support PCIe interface in U-Boot

7.2 Ethernet support

The Ethernet configuration is specified in the board DTS files in gmac_cfg.

The IPQ5018 SoC has 2 GMAC controllers.

- GMAC0 is connected to internal PHY (GEPHY)
- GMAC1 is connected to external PHY like QCA8081, QCA8033, and S17C switch via UNIPHY.

The GMAC0 and GMAC1 controllers specify the following in the DTS file:

- Unit ID
- Base address

Example: GMAC 0 and GMAC 1

```
gmac1_cfg {
   unit = <0>;
   base = <0x39C00000>;
};

gmac2_cfg {
   unit = <1>;
   base = <0x39D00000>;
};
```

7.2.1 GEPHY configuration

The GEPHY is an internal PHY interface which is directly connected to the GMAC0 controller. The GEPHY interface configures SGMII, and the PHY address specifies the following in the DTS file as given below.

```
gmac1_cfg {
   unit = <0>;         /* GMAC Unit id */
   base = <0x39C00000>;      /* GMAC0 Base address */
   phy_address = <7>;     /* GEPHY PHY address */
};
```

7.2.2 UNIPHY configuration

The UNIPHY interfaces configure the PHY/Switch connected to the GMAC1 controller based on the supported PHY mode.

IPQ5018 SoC UNIPHY interfaces support the operating modes listed here.

PHY / Switch	Supported PHY mode
QCA8081	SGMII, SGMII+
QCA8033	SGMII
S17C	SGMII

Table 7-1 U-Boot-2016 supported configurations for QTI reference boards

UNIPHY0 + GEPHY usecase	UNIPHY0 mode	GEPHY mode
1:SGMII + 1: SGMII – MP03.1	SGMII	SGMII
1:SGMII + 1: SGMII+ – MP03.3	SGMII+	SGMII
1:SGMII + 1: SGMII – MP02	SGMII	SGMII

7.2.3 MP0x U-Boot support for QCA8081, QCA8033 PHYs and S17C switch

For QCA8081 and QCA8033 PHYs, specify the following in the DTS file:

- PHY address
- Reset GPIO number

For S17C switch, specify the following in DTS file:

- MAC power
- Enable
- Port count
- PHY address
- Reset GPIO number

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Example: AP-MP03.3, QCA8081 is connected to UNIPHY and uses GPIO 39 as reset pin

```
gmac2_cfg {
    phy_address = <0x1c>;
    napa_gpio = <39>;
};
```

Example: AP-MP02.1, QCA8033 is connected to UNIPHY and uses GPIO 26 as reset pin

```
gmac2_cfg {
     phy_address = <0x4>;
     8033_gpio = <26>;
};
```

Example: AP-MP03.1, S17C switch is connected to UNIPHY and uses GPIO 39 as reset pin

```
gmac2_cfg {
      mac_pwr = <0xaa545>;
      s17c_switch_enable = <1>;
      switch_port_count = <4>;
      switch_phy_address = <0 1 2 3>;
      switch_gpio = <39>;
};
```

7.3 Tiny U-Boot

Tiny U-Boot is a minimum version of U-Boot which supports only SPI-NOR flash. The following features are *removed* from the actual U-Boot:

- NAND Flash
- eMMC Flash
- PCIe
- USB
- I2C
- Commands
  - Memtest
  - UBI
  - Cache-related commands
- GZIP feature

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.4 Disable unnecessary flash types and peripherals in U-Boot

Flash

By default, all flash types are initialized in U-Boot. Follow these to disable the unnecessary flash types:

Flash Type	To disable the flash type if it is not needed:
NOR	Remove these lines in qsdk/qca/src/u-boot-2016/include/configs/ipq5018.h: ■ #define CONFIG_QCA_SPI ■ #define CONFIG_SPI_FLASH ■ #define CONFIG_CMD_SF ■ #define CONFIG_SPI_FLASH_STMICRO ■ #define CONFIG_SPI_FLASH_WINBOND ■ #define CONFIG_SPI_FLASH_MACRONIX ■ #define CONFIG_SPI_FLASH_GIGADEVICE ■ #define CONFIG_SPI_FLASH_SPANSION ■ #define CONFIG_SF_DEFAULT_BUS 0 ■ #define CONFIG_SF_DEFAULT_CS 0 ■ #define CONFIG_SF_DEFAULT_MODE SPI_MODE_0 ■ #define CONFIG_SF_DEFAULT_SPEED (48 * 1000 * 1000) ■ #define CONFIG_SPI_FLASH_BAR 1 ■ #define CONFIG_SPI_FLASH_USE_4K_SECTORS ■ #define CONFIG_ENV_IS_IN_SPI_FLASH 1
NAND	Remove the nand node in qsdk/qca/src/u-boot-2016/arch/arm/dts/ipq5018-soc.dtsi
eMMC	Remove this line in qsdk/qca/src/u-boot-2016/configs/ipq5018_defconfig: ■ CONFIG_MMC_FLASH=y

Peripherals

By default, these peripherals are enabled. Follow these steps to disable the unnecessary peripherals on the board:

Peripheral	To disable the peripheral if it is not needed:
USB	Remove these lines in qsdk/qca/src/u-boot-2016/configs/ipq5018_defconfig: ■ CONFIG_USB=y ■ CONFIG_DM_USB=y ■ CONFIG_USB_XHCI_IPQ=y
PCIe	Remove this line in qsdk/qca/src/u-boot-2016/include/configs/ipq5018.h: ■ #definePCI_MAX_DEVICES 1 Remove this line in qsdk/qca/src/u-boot-2016/configs/ipq5018_defconfig: ■ CONFIG_PCI_IPQ=y
I2C	Remove the node i2c0 in qsdk/qca/src/u-boot-2016/arch/arm/dts/ipq5018-mp*.dts

7.5 Machine ID

Machine ID (machid) is used in U-Boot to differentiate between boards. The following machids are supported in QSDK builds for IPQ5018.

Board	Machid	Flash/DDR
AP-MP02.1	8040000	NOR flash DDR3
AP-MP03.1	8040001	NAND flash DDR3
AP-MP03.1-C2	8040101	eMMC flash DRR3
AP-MP03.3	8040002	NAND flash DDR3
AP-MP03.3-C2	8040102	eMMC flash DDR3

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8 Combined UBI image for NAND flash

Separate partitions are not used for Wi-Fi and Bluetooth firmware images in NAND and NORplusNAND flash layout for IPQ5018. NAND flash layout is updated to include Wi-Fi and Bluetooth firmware images as individual UBI volumes along with Kernel and Rootfs volumes.

For NOR/eMMC flashes, there are separate partitions for Wi-Fi and Bluetooth firmware images.

Flash commands are added to update the UBI volume specifically.

```
flash ubi_vol_name
```

Example: `flash wifi_fw`

The updated flash commands are listed here.

Image	Combined UBI image flash layout
kernel	`tftpb openwrt-ipq-ipq50xx-qcom-ipq50xx-mpxx-fit-uImage.itb` `flash kernel`
rootfs	`tftpb openwrt-ipq-ipq50xx-squashfs-root.img` `flash kernel`

Wi-Fi FW	`tftpb wifi_fw_squashfs.img`
	`(or)`
	`tftpb wifi_fw_ipq5018_qcn9000_squashfs.img`
	`flash wifi_fw`
	NOTE *wifi_fw_squashfs.img* is IPQ50xx internal radio firmware and *wifi_fw_ipq5018_qcn9000_squashfs.img* is IPQ50xx internal radio + QCN9000 firmware

BT FW	`tftpb bt_fw_patch_squashfs.img` `flash bt_fw`

NOTE wifi_fw_ubi.img is no longer used with a NAND flash layout.

While generating a single-image using pack script, combined UBI image (kernel + rootfs + Wi-Fi firmware + BT firmware) is generated with the ubinize configuration. See the following sample configuration available at /ipq/ipq5018/flash_partition/ipq5018-ubinize*.cfg.

```
[kernel]
# Volume mode (other option is static) mode=ubi
# Source image
image=openwrt-ipq-ipq50xx-qcom-ipq50xx-mpxx-fit-uImage.itb
vol_size=6MiB vol_type=static
# Volume ID in UBI image
vol_id=0
# Volume name vol_name=kernel

[wifi_fw]
# Volume mode (other option is static) mode=ubi
# Source image image=wifi_fw_squashfs.img vol_size=6MiB vol_type=static
# Volume ID in UBI image vol_id=1
# Volume name vol_name=wifi_fw

[bt_fw]
# Volume mode (other option is static) mode=ubi
# Source image image=bt_fw_patch_squashfs.img vol_size=256KiB vol_type=static
# Volume ID in UBI image vol_id=2
# Volume name vol_name=bt_fw

[rootfs]
# Volume mode (other option is static) mode=ubi
# Source image
image=openwrt-ipq-ipq50xx-squashfs-root.img
# Volume ID in UBI image vol_id=3
# Allow for dynamic resize vol_type=dynamic
# Volume name vol_name=ubi_rootfs

[rootfs_data] mode=ubi
# this is arbitrary, since autoresize below will use up the remaining
# free space on the mtd volume vol_size=1KiB
vol_id=4 vol_type=dynamic vol_name=rootfs_data vol_flags=autoresize
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9 Include QCN90xx firmware image

By default, MP03.1 and MP03.3 RDPs has the firmware image for both internal Q6 and QCN9000.

Wi-Fi firmware image	Description
wifi_fw_squashfs.img	Firmware image for internal WCSS Q6.
wifi_fw_ipq5018_qcn9000_squashfs.img	Firmware image for both internal Q6 and QCN90xx.

The ubinize configuration file can be updated directly to include Wi-Fi firmware with the QCN90xx binaries as given here.

Path:/ipq/ipq5018/flash_partition/ipq5018-ubinize*.cfg [wifi_fw]

Volume mode (other option is static)

```
mode=ubi
```

Source image

```
image=wifi_fw_ipq5018_qcn9000_squashfs.img
vol_size=6MiB
vol_type=static
```

Volume ID in UBI image

```
vol_id=1
```

Volume name

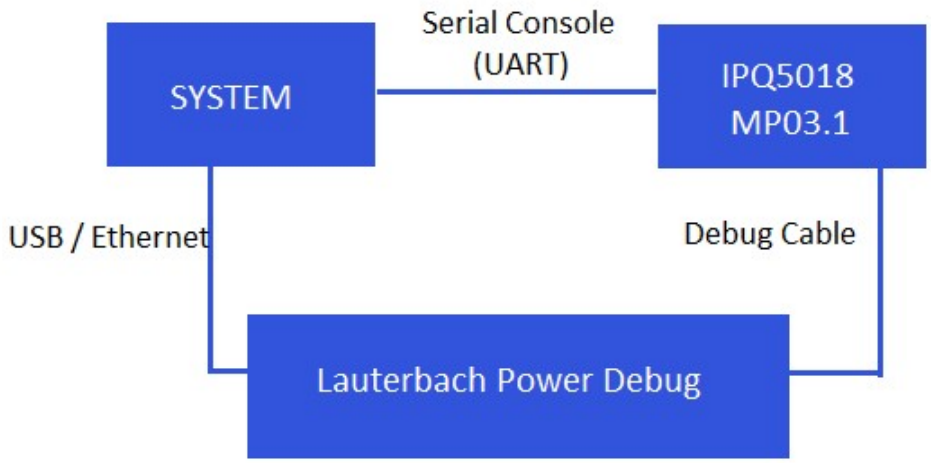
```
vol_name=wifi_fw
```

10 Debug interfaces

All debug interfaces for the IPQ50xx SoC are accessible to an external debugger through a standard JTAG connector. QTI recommends using the TRACE32 in-circuit debugger from Lauterbach that supports the Cortex-A53/QDSP6 debugging such as:

- Lauterbach LA-7705: Power debug module Ethernet
- Lauterbach LA-7746: JTAG debugger for ARM7 20-pin connector (ICD)
- Lauterbach LA-3741A: JTAG debugger extension for Hexagon DSP
- Lauterbach LA-3743X: JTAG debugger extension for Arm V8
- Lauterbach TRACE32 software version should be R.2020.09.000128638 or greater

An example of Lauterbach topology for an IPQ5018 AP.MP03.1 platform is shown here.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.1 Set up Lauterbach

The Lauterbach debugger is an advanced tool recommended for debug and image flash on the IPQ50xx reference platform. It provides detailed information about CPU register dump and symbol details in line with source code. The 20-pin JTAG header on the IPQ5018 platform is mounted to externally connect the Lauterbach debugger. The script file **<*File*>.cmm** uses one cmm-style command on each line.

1. In TRACE32, select **File > Run Batchfile.**
2. On the TRACE32 command line, enter:

```
DO file.cmm
```

OR

```
CD.DO file.cmm
```

10.2 TRACE32 scripts

The TRACE32 package is a part of QSDK. The git project reference is **system/tools/jtag-scripts**. This package contains the core start and recovery scripts. Its folder contents are:

```
jtag-scripts
| - IPQ5018
|     | - Maple_start.ts2
|     | - core
|     |     |  - A53
|     |     |     | - Maple_AR_DAP_CortexA53_Cluster0_Core0.cmm
|     |     |     | - Maple_AR_DAP_CortexA53_Cluster0_Core1.cmm
|     |     |     | - Maple_AR_DAP_CortexA53_Cluster0_Core01.cmm
|     |     |  - QDSP6_WCSS
|     |     |     | - AR_JTAG_Q6_WCSS.cmm
|     |     |  - UBI32
|     |     |     | - MP_UBI32_0.cmm
```

10.3 Debug using JTAG

The debug procedures using JTAG are described in the following sections:

- Disable CONFIG_PERF_EVENTS when using JTAG debug
- Launch T32
- Set up kernel debug
- Debug modules
- Multicore debug

10.3.1 Disable CONFIG_PERF_EVENTS when using JTAG debug

- Disable CONFIG_PERF_EVENTS

  The kernel resets the Arm debug module while booting if CONFIG_HAVE_HW_BREAKPOINT is enabled. To avoid this behavior, disable CONFIG_PERF_EVENTS; this action selects CONFIG_HAVE_HW_BREAKPOINT.

- Disable from CONFIG_PERF_EVENTS config in target/linux/ipq/config-4.4:

```
diff --git a/target/linux/ipq/config-4.4 b/target/linux/ipq/config-4.4
index f7556de..12ff190 100644
--- a/target/linux/ipq/config-4.4
+++ b/target/linux/ipq/config-4.4
@@ -465,7 +465,7 @@ CONFIG_PCIE_QCOM=y
CONFIG_PCI_DOMAINS=y
CONFIG_PCI_DOMAINS_GENERIC=y
CONFIG_PCI_MSI=y
-CONFIG_PERF_EVENTS=y
+# CONFIG_PERF_EVENTS is not set
CONFIG_PERF_USE_VMALLOC=y
CONFIG_PGTABLE_LEVELS=2
CONFIG_PHYLIB=y
```

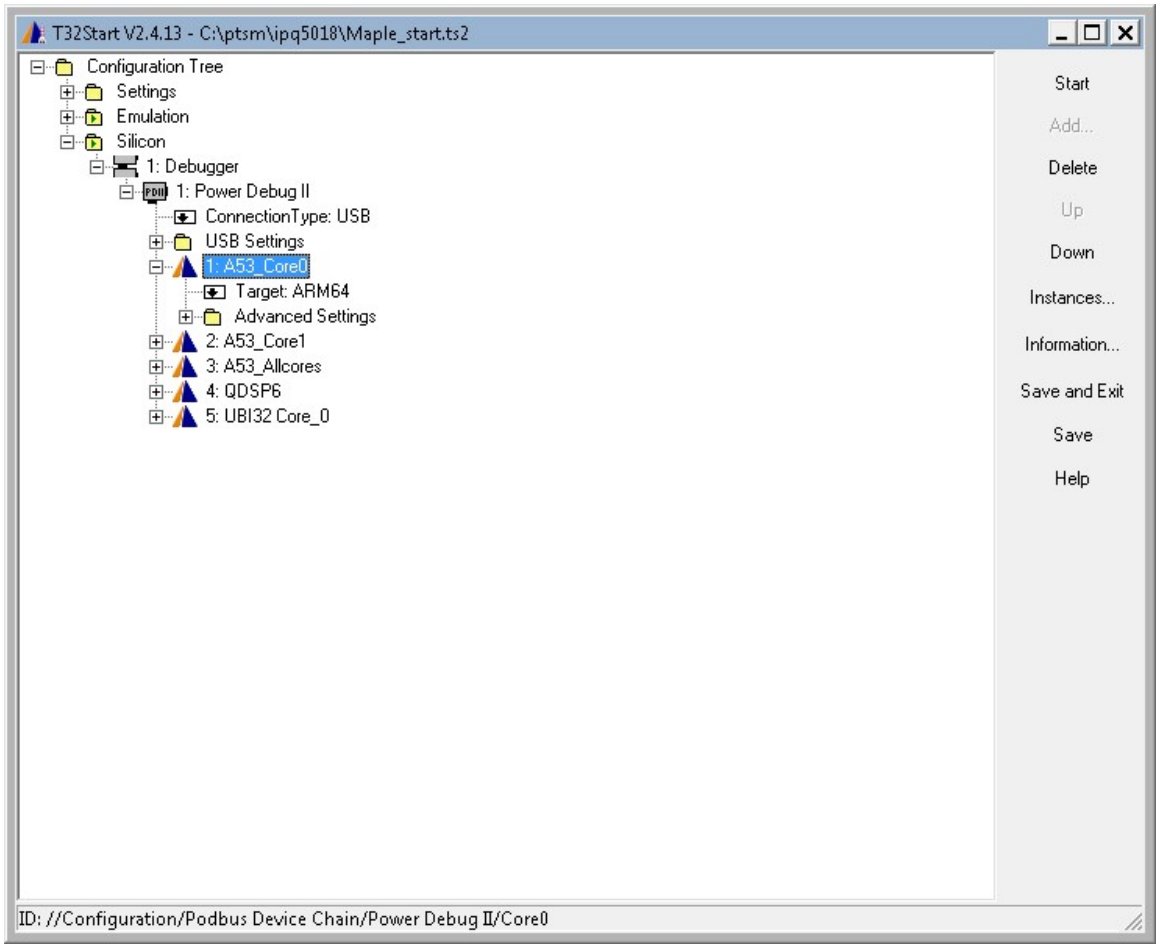
- Disable CONFIG_PACKAGE_PERF from OpenWrt configuration:

```
make menuconfig
        Development  --->
         < > perf.......................... Linux performance monitoring
tool
```

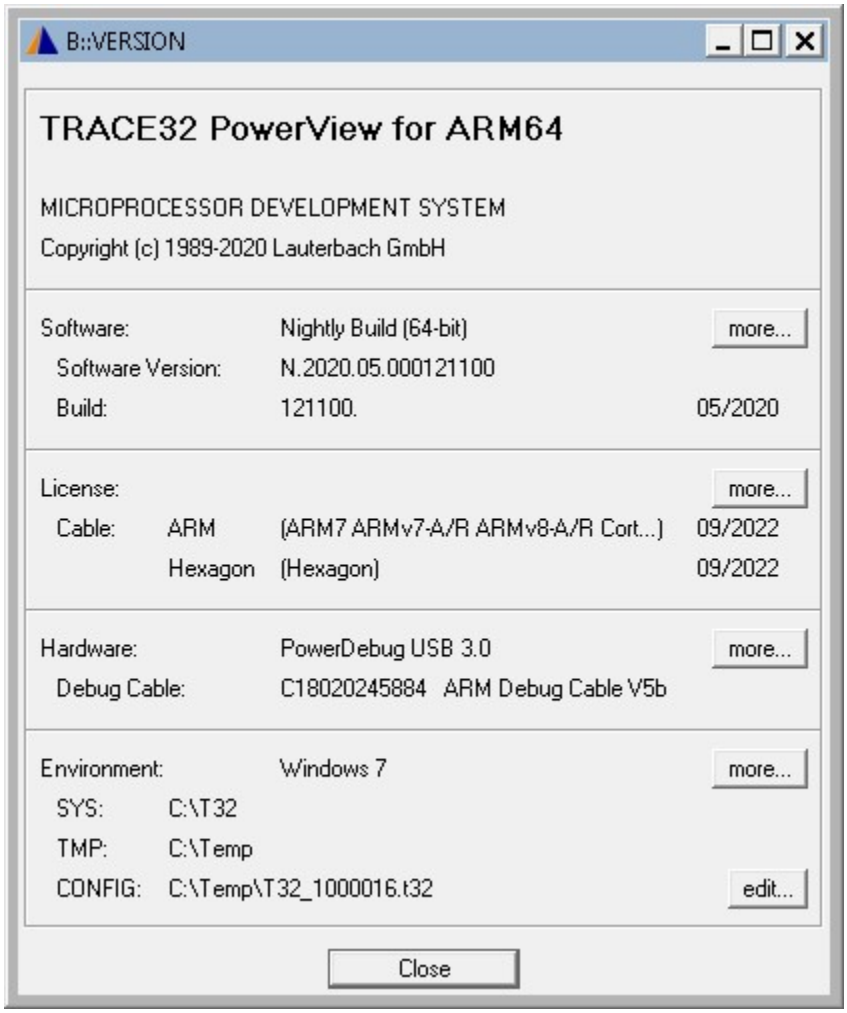
10.3.2 Launch T32

The IPQ5018-specific TRACE32 scripts and T32 launch configuration files are available at ./common/t32/ipq5018/

1. Double-click **Maple_start.ts2** to launch an IPQ5018 TRACE32 session.



2. Select or double click on **Core0** and click **Start** to launch TRACE32 for CPU Core0. The required software version of TRACE32 is **N.2020.05.000121100**.



10.3.3 Set up kernel debug

Disable the kernel software watchdog and RCU mechanism of stall detection:

```
IPQ5018# setenv bootargs "console=ttyMSM0,115200n8 nowatchdog
rcupdate.rcu_cpu_stall_suppress=1"
IPQ5018# saveenv
    Saving Environment to NAND...
    Erasing NAND...
    Erasing at 0xd20000 -- 100% complete.
    Writing to NAND... OK
IPQ5018# bootipq
```

Conversely, it can be set after kernel boots also:

```
root@OpenWrt:/# echo 0 > /proc/sys/kernel/watchdog
root@OpenWrt:/# echo 1 > /sys/module/rcupdate/parameters/
rcu_cpu_stall_suppress
```

In the TRACE32 command window, use these commands:

1. Use the following commands in the T32 command window:

   ```
   sys.attach
   break
   ```

2. Disable the nonsecure and secure hardware watchdog.

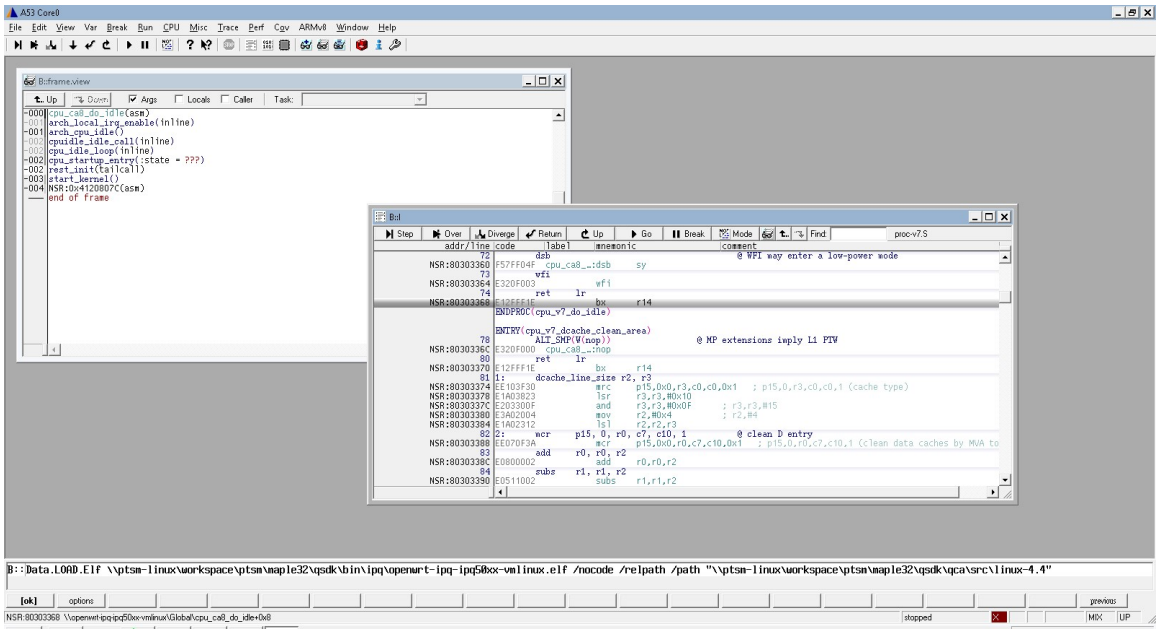
   If there is a delay, watchdog expires and resets the system.

   ```
   d.s AZSR:0xb017008 %LE %Long 0x0
   d.s AZSR:0x4AA004  %LE %Long 0x0
   go
   ```

3. Load the vmlinux symbol from qsdk\bin\ipq\openwrt-ipq-ipq50xx-vmlinux.elf:

   ```
   d.load.elf <path> qsdk\bin\ipq\openwrt-ipq-ipq50xx-vmlinux.elf /nocode /
   relpath /path "<path to linux kernel source code>"
   ```

   Users can now debug the kernel using breakpoints, stop, single step, and examine data structures.



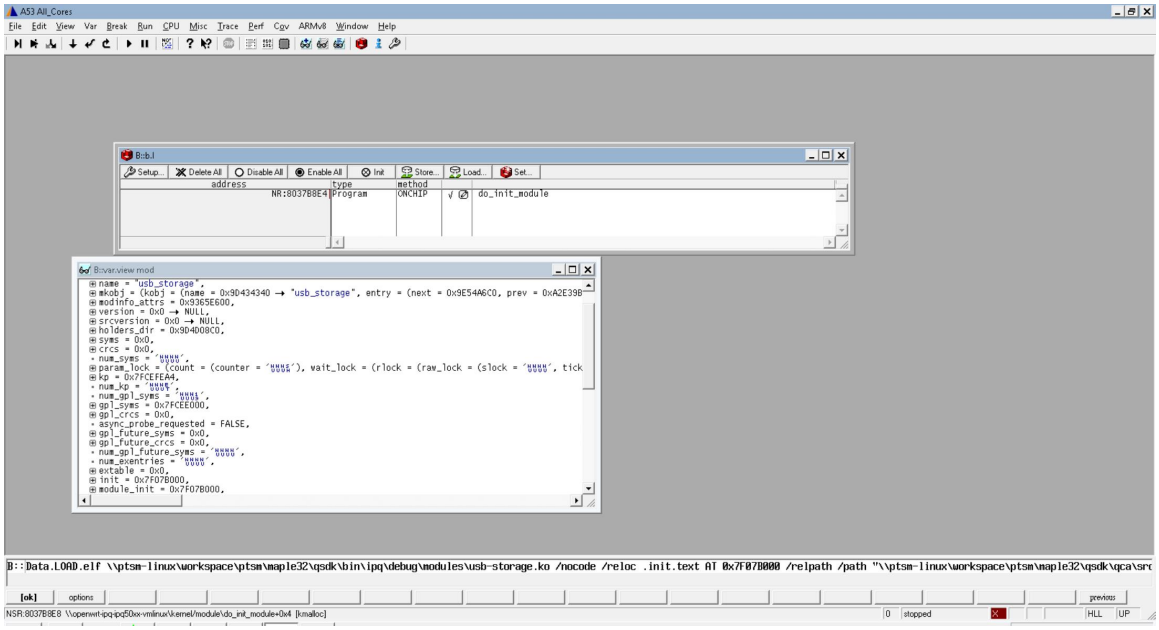
10.3.4 Debug modules

To debug modules from initialization, attach them to the kernel before they insmod:

1. Load kernel symbols before insmod.
2. Put a breakpoint at do_init_module() using:

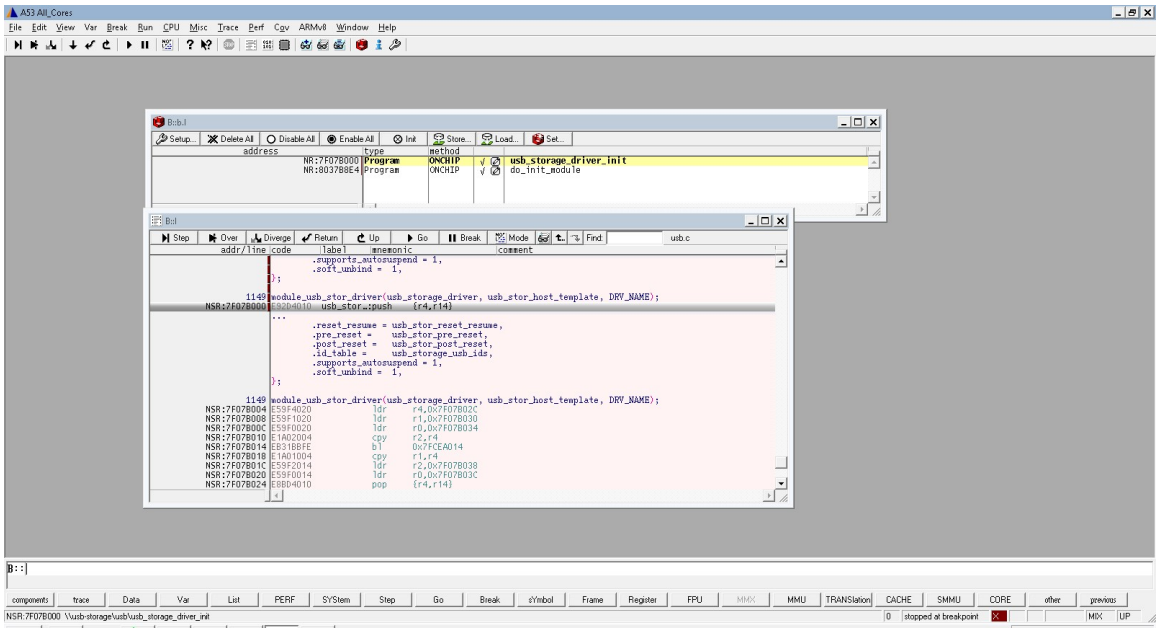
   ```
   b.s do_init_module /onchip
   ```

3. At every breakpoint, hit at do_init_module().
4. To view the argument for do_init_module:



5. When <mod> name matches the module to debug, load the module symbols:

```
Data.LOAD.elf <path>\qsdk\bin\ipq\debug\modules\<module.ko> /NOCODE /
NOCLEAR /RELOC .init.text AT <address shown in mod->module_init> /
RELOC .text AT <address show in mod->module_core> /RELPATH /PATH "<module
code path>"
```



Users can now set the breakpoint and step through any function of the loaded module.

10.3.5 Multicore debug

1. Select individual cores (A53_core0, A53_core1) and click **Start** for each core.
2. Use the following commands in any of the core windows to disable hardware watchdog.

```
break
d.s AZSR:0x4AA004  %LE %Long 0x0
d.s AZSR:0xb017008 %LE %Long 0x0"
go
```

3. Begin debugging.
   - Load kernel image (Vmlinux) symbols and module symbols as required in all TRACE32 windows.
   - Set breakpoint in all the core windows.
   - Check the window in which the breakpoint is hit and view registers or stack frame:

   The procedure is as same as for Core0. On-chip breakpoints are effective only on the core where they are placed. Thus, when debugging the SMP kernel or kernel modules, set the on-chip breakpoints in all the core TRACE32 windows.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

11 Flash image update procedure and non-bootable board recovery

The software image in a normal booting board can be updated using several methods, described in section Options to flash the software image.

If the image update fails (for example, due to a power failure) and the board will not boot, recovery may be possible using one of the methods described in Section Recover using JTAG and Section Board recovery using USB emergency download mode.

Additionally, if a board supports multiple boot memory technologies (For example: MP03.1 can boot from any of the NOR, SPI-NAND or eMMC memories that are provisioned on the board), flashing a known working image into any or all of the memories not used for booting/testing allows the user to simply change the boot memory selection via DIP switch, then reboot and reflash the memory that contains the non-booting image.

11.1 Options to flash the software image

The following sections describe alternate methods to transfer an image to the board; and selection of an appropriate image. If a board is provisioned with multiple bootable flash memory types (for example, MP03.3 can boot from any of NOR, eMMC, or SPI-NAND), the image in the other memories can be updated simply by using the correct file as shown in *Image selection*.

11.1.1 Image selection

The following images are generated in the ./common/build/bin directory by the last step of the QSDK build process (python update_common_info.py):

Flash type	Single image name (32 bit)	Single image name (64 bit)
NOR	nor-ipq5018-single.img	nor-ipq5018_64-single.img
NAND	nand-ipq5018-single.img	nand-ipq5018_64-single.img
eMMC	emmc-ipq5018-single.img	emmc-ipq5018_64-single.img
NOR plus NAND	norplusnand-ipq5018-single.img	norplusnand-ipq5018_64-single.img
NOR plus EMMC	norplusemmc-ipq5018-single.img	norplusemmc-ipq5018_64-single.img
Tiny-NOR single image	tiny-nor-ipq5018-single.img	tiny-nor-ipq5018_64-single.img

11.1.2 Transfer the image using TFTP

1. Set up the TFTP server. The IP address of the ethernet port that will connect to the MP0x board must be configured manually.
2. Copy the single image to flash from common\build\bin to the TFTP server root folder.
3. In U-Boot, set the board IP address and the server IP address which were configured in step 1:

   ```
   setenv ipaddr <board IP address>
   setenv serverip <server IP address>
   ping $serverip
   ```

   When a 'server is alive' response is printed, proceed to the next step.
4. Transfer the image from the TFTP server to MP0x board using the tftpboot command:

   ```
   tftpboot 0x44000000 <image-name>
   ```

   where <image-name> is selected as appropriate from the image selection section.

11.1.3 Transfer the image using USB (host mode)

This feature is not enabled in U-Boot by default. Enable the configuration as shown in the Enabling U-boot USB (host mode) section (note that this requires reflashing of the U-boot image at least using tftpboot, JTAG or Emergency Download mode for the image transfer), and execute the following commands:

1. Format a USB disk as FAT32.
2. Copy the image to be flashed to USB disk.
3. Connect to port 0.
4. Start USB from U-Boot prompt:

   ```
   usb start
   ```
5. Load the image from disk:

   For USB port 1:

   ```
   fatload usb 0:1 0x44000000 <Image Name>
   ```

11.1.4 Reference kit

The IPQ50xx AP.MP03 reference design kit includes:

- IPQ50xx AP.MP03 reference design
- 12 V power adapter
- RS232 UART adapter

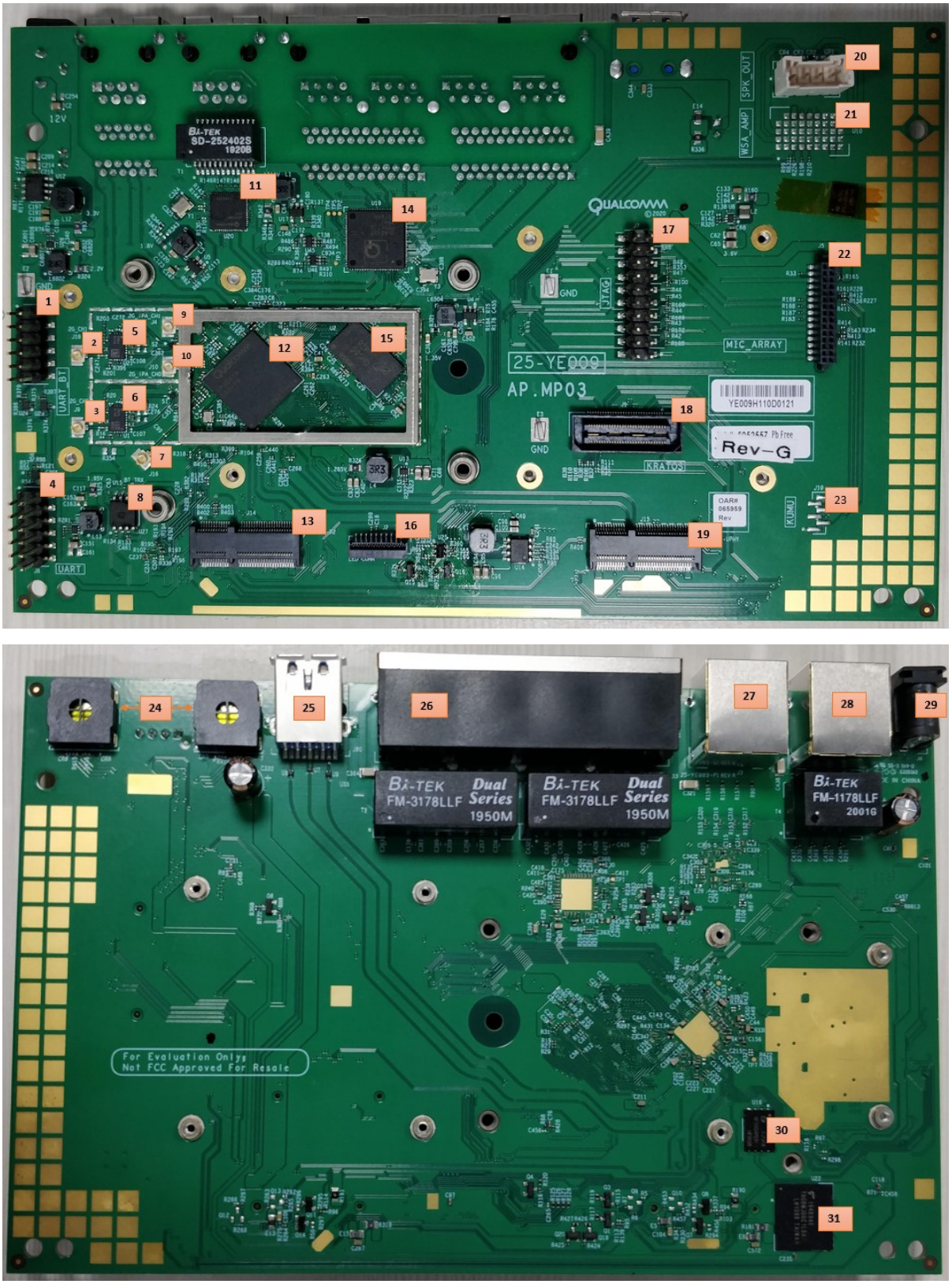


Figure 11-1 AP.MP03 board

Table 11-1 IPQ50xx AP.MP03 interface details

Item	Description
1	BT HCI UART
2	XFEM CH 0 UFL
3	XFEM CH 1 UFL
4	Console UART
5	XFEM CH 0
6	XFEM CH 1
7	BT UFL
8	NOR flash
9	IPA CH 0 UFL
10	IPA CH 1 UFL
11	QCA8081 PHY
12	IPQ5018
13	PCIe 0 – X2
14	QCA8337N – S17C
15	DDR3
16	LED connector
17	JTAG DBG
18	Kratos connector
19	PCIe 1 – X1
20	Speaker connector
21	WSA 8815 audio module
22	Circular MIC array connector
23	BLSP1 UART
24	8 Ohm speaker
25	USB connector
26	S17C 4x 1 GB port
27	QCA8081 port
28	INT 1G GEPHY 12 V DC 12 V DC
29	12 V DC QSPI NAND
30	QSPI NAND
31	eMMC

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

11.1.5 Flash the image and perform a hard reboot

Execute this command at the U-Boot prompt after the image has been transferred to the board DDR memory using one of the methods described in Sections Transfer the image using TFTP, Transfer the image using USB (host mode), Recover using JTAG, or Board recovery using USB emergency download mode to flash the image and perform a hard reboot after flashing completes by power-cycling the board:

```
imgaddr=0x44000000
source $imgaddr:script
```

Do not power-cycle the board before flashing is complete; otherwise image corruption could brick the board.

Completion of the flashing command is indicated by re-print of the command line prompt: this may take several minutes depending on memory technology and image size.

11.1.6 Sysupgrade support

A feature is available to upgrade images from the OpenWrt web interface without the need for a TFTP server. After using U-Boot to flash an initial image and booting the device, any future upgrades can be done from the web interface. The complete flash image can be upgraded using the single image file generated by the update_common_info.py script.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The upgrade of images using the OpenWrt Web interface process takes several minutes to complete, up to a maximum of approximately five minutes, depending on various factors, such as memory technology, vendor, image size, browser connectivity, and network load. If system power is lost during this period, or if a key-press event is detected on the serial console port, the upgrade process is interrupted, and an invalid image remains in the flash memory. Completion of flash upgrade process is signaled by the refresh of the OpenWrt login page, and lack of further messages on the serial console.

For information on fail-safe image upgrade support, see *Fail Safe Boot for 802.11ac and 802.11ax WLAN AP Chipsets Application Note* (80-YA809-1).

If a board is configured for failsafe boot, and the U-Boot environment variable 'bootargs' contains any reference to the rootfs or other partitions managed by the failsafe scheme, the sysupgrade process may only be partially successful. The board may or may not boot subsequently, and the image may only be partly updated.

The sysupgrade command may also be used from the Linux console. This requires the sysupgrade image to be visible within the Linux filesystem that is supplied via development host SSH or tftpd server; or visible on an NFS server that is mounted to the local file system; or transfer via USB storage device.

The sysupgrade feature can only be used to update the image in the memory that the board booted from and is used by running kernel. If the board supports multiple memory technologies, use U-Boot to change the image in memories that the board did not boot from.

11.1.7 Sysupgrade support with eMMC GPT partition update

To change the partition layout in eMMC sysupgrade, update GPT and GPT backup partition by uncommenting do_flash_gpt() function call. This change provides support for future sysupgrade from Linux v4.4 to v5.4 with the removal of rootfs_data partition.

In the code, uncomment the following function call in *target/linux/ipq/lib/upgrade/platform.sh*:

```
- # do_flash_gpt
+ do_flash_gpt
for sec in $(print_sections $1); do
```

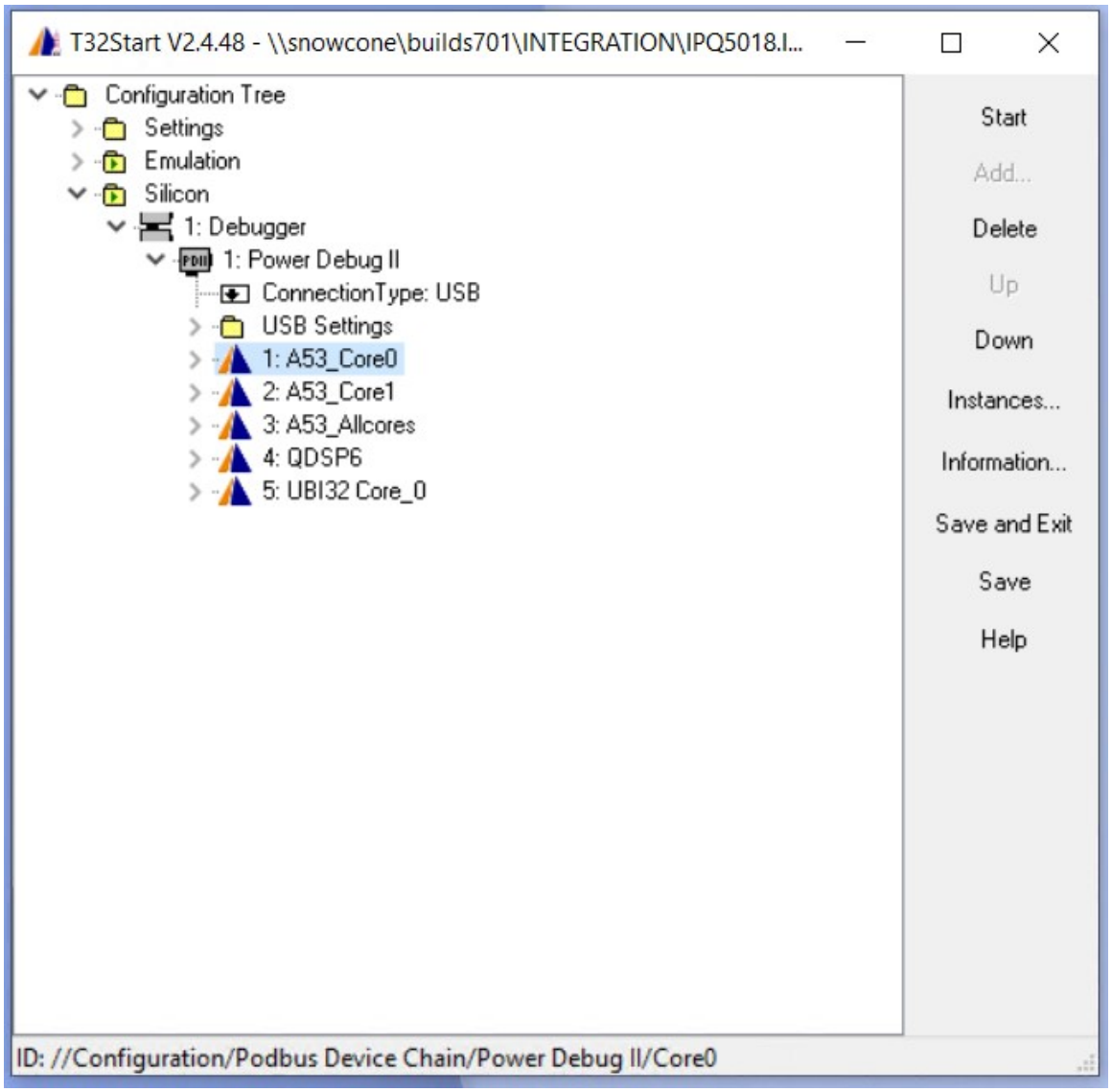
11.2 Recover using JTAG

The JTAG board recovery procedure uses a Lauterbach T32 probe to boot the board and load a U-boot image to DDR. After executing the U-boot, the flash image can be transferred and flashed using the steps outlined in Section *Transfer the image using TFTP, Transfer the image using TFTP* or Section *Transfer the image using USB (host mode), Transfer the image using USB (host mode),* followed by Section *Flash the image and perform a hard reboot, Flash the image and perform a hard reboot*. Ensure that both the T32 probe and evaluation board serial console are connected to the debug host PC to complete this procedure.

1. Before running recovery, place all the files (CDT file, SBL_flashless.mbn, and U-boot.elf) in a directory that the T32 jtag_recovery script can read (select image path as shown). The CDT file is chosen (see following screenshot) based on the board by steps in:

   ```
   ./IPQ5018.ILQ.11.3/common/t32/ipq5018/recovery/jtag_recovery.cmm
   ```

2. Start TRACE32. Double-click and select Maple_start.ts2. The TRACE32 window appears showing all cores:

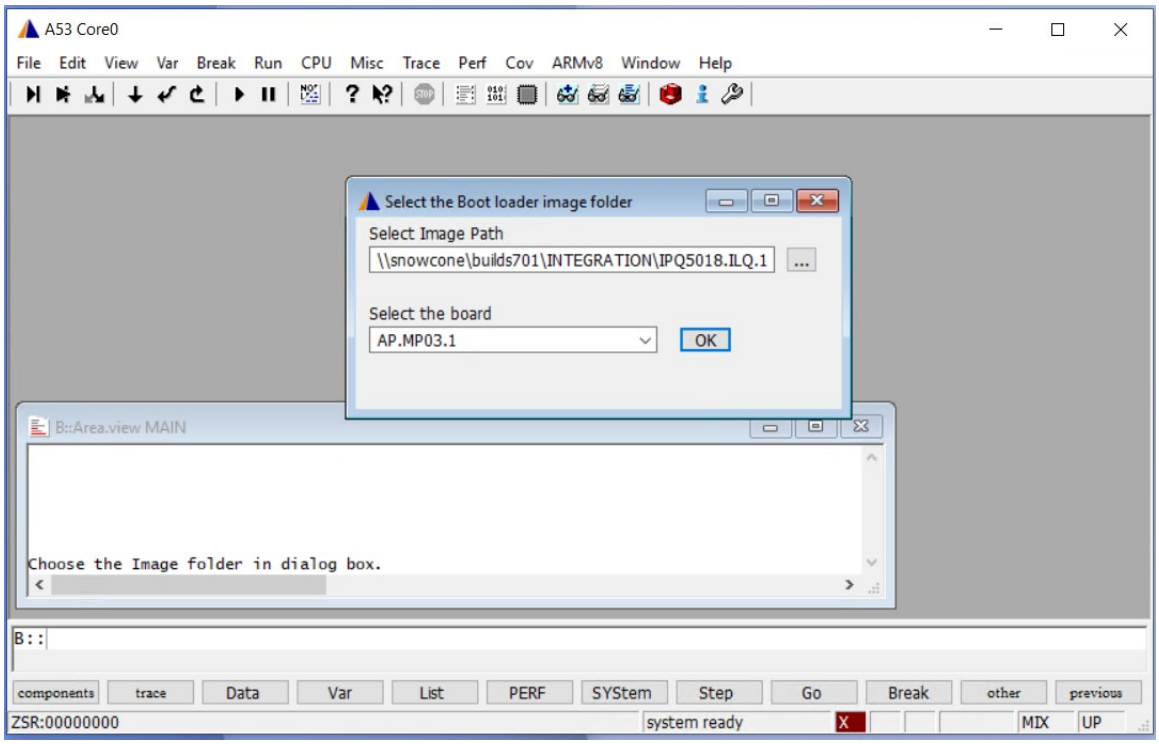


3. In T32Start window, select **A53 Core_0** and Click **Start**. A new Core0 window appears.
4. Start the recovery. In the Core0 window, run the following commands:

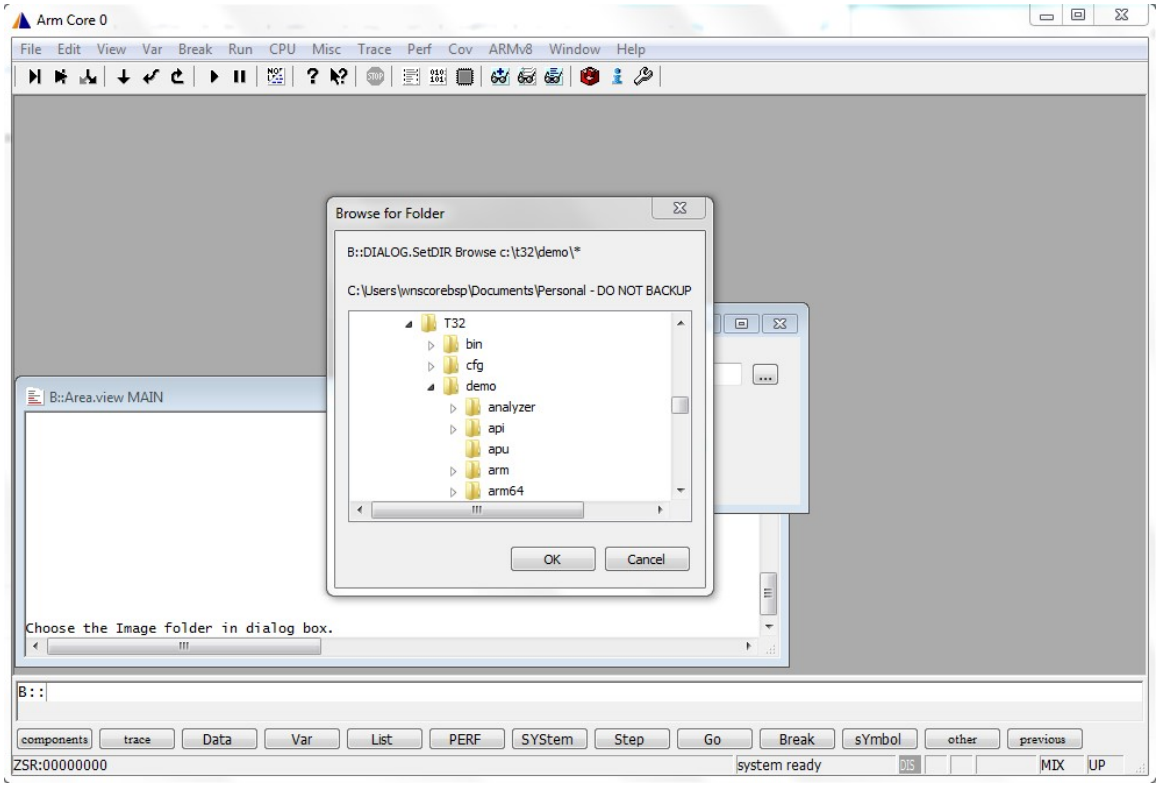
   ```
   cd.do jtag-scripts/ipq5018/recovery/jtag_recovery.cmm
   ```

   The source script is available at ipq5018/recovery/jtag_recovery.cmm.

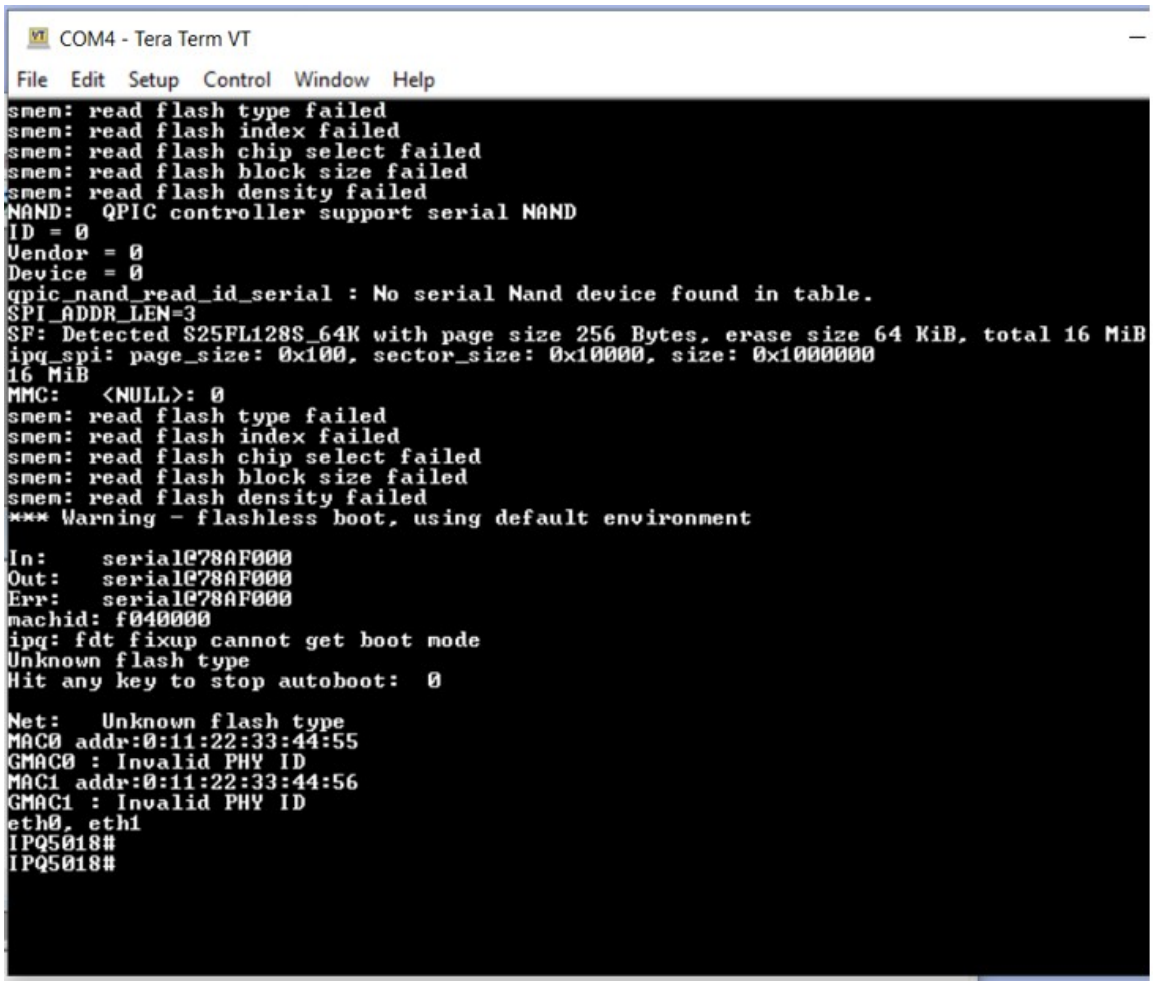
The following window appears to capture further details:



5. Select the folder that contains the DDR SBL (**sbl1_flashless.mbn**) and U-Boot images (**openwrt-ipq5018-u-boot.elf**):



6. Select the correct reference board and click **OK**. Bootlog messages appear in the serial console window if the CPU boots:



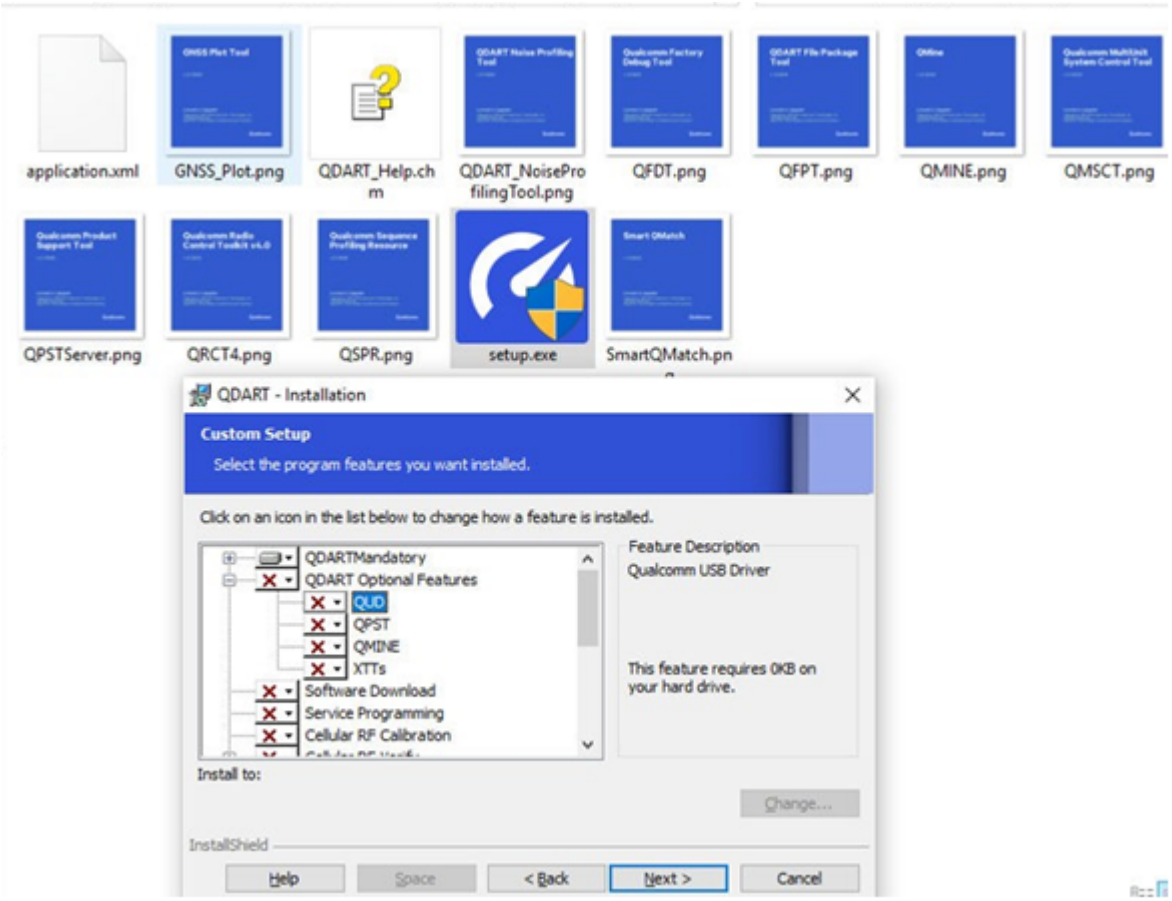
NOTE This procedure immediately runs the initialize clocks and DDR functions, and then loads and runs U-Boot. An error may occur if there is no kernel image in the flash device. Stop auto kernel load and boot by pressing any key on the keyboard that is attached to the MP0x serial console (terminal emulation application).

NOTE Select and flash the complete image as described in Section *Transfer the image using TFTP* or Section *Transfer the image using USB (host mode)*, followed by Section *Flash the image and perform a hard reboot*. Power down the board and remove the T32 probe after image flashing is complete and before powering up to test the new image.

11.3 Board recovery using USB emergency download mode

The customers who are licensed to access the tools package can install the required USB drivers from the QDART tools package. Download the QDART installer package from the CreatePoint Tools page, run the installer in custom mode, and install Qualcomm USB Driver (QUD) as follows.

For details on the latest QDART version, see the corresponding QCA_Networking_2021.SPF.x.x Release Notes.



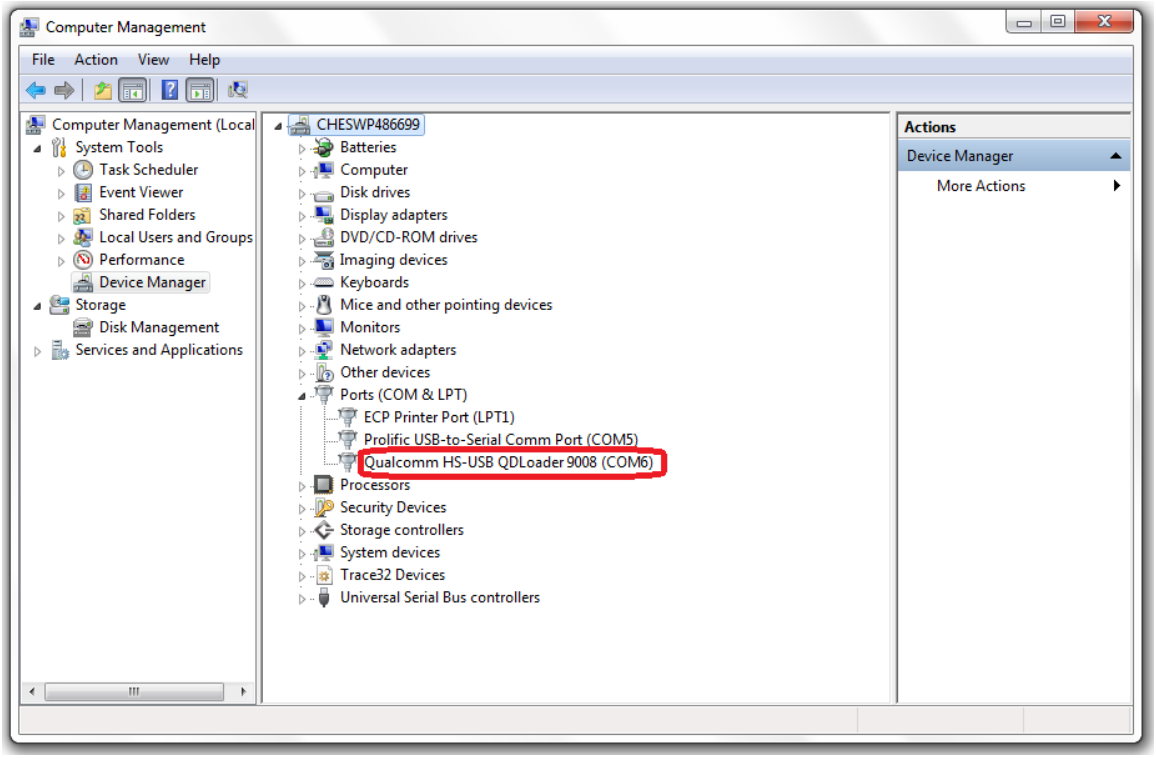
Connect the USB port 0 and the PC using an A-to-A cable and a Type-C converter. Install the necessary drivers in Windows (if it is not already done). The board enters USB download mode in two cases:

- If the Primary Bootloader cannot detect a valid SBL image
- Boot from USB bootstrap is enabled

Recover using USB:

1. Ensure that both the MP0x serial console and USB port 0 connections are made and power on the board.

   If it is the first time after installing drivers in PC, wait for the drivers to load. The Qualcomm HS-USB device appears in the device manager.



2. Create a folder on the PC acting as USB host (for example, called **ipq**) and copy the following files into it from the QSDK build server:
   - sbl1_flashless.mbn
   - CDT file for the board
   - tz.mbn
   - devcfg_noac.mbn
   - openwrt-ipq5018-u-boot.mbn
   - QSaharaServer.exe
3. Power cycle the target by removing both power cable and USB cable, re-connect USB, power (in the order), immediately run either of these following commands from the folder created in Step 2 (this example uses the name **ipq**):
   - If the boot from USB option is selected:

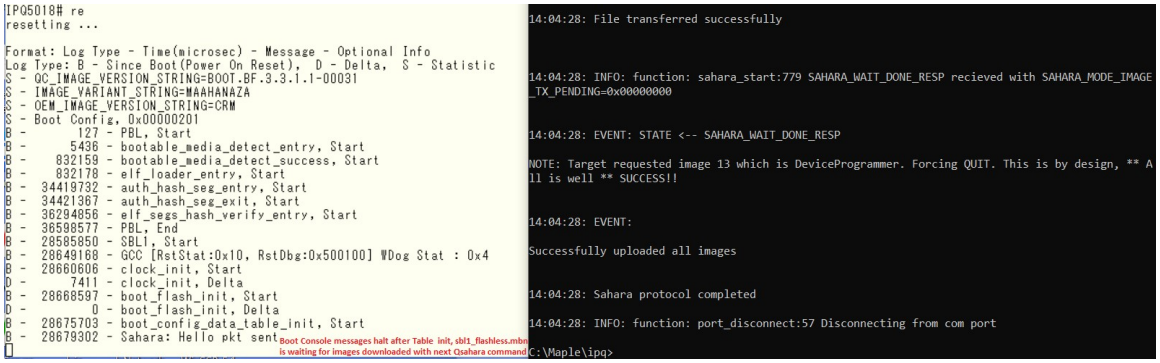
     ```
     QSaharaServer.exe -p \\.\COM<port number> -s 21:sbl1_flashless.mbn -v 3
     ```

   - If the board was bricked (flash does not have a valid booting image),

     ```
     QSaharaServer.exe -p \\.\COM<port number> -s 13:sbl1_flashless.mbn -v 3
     ```

Where <port number> is the com port number detected on Host PC. See the Device Manager for details.

The following screenshot shows the console state on DUT and Host PC after sbl1_flashless.mbn has been correctly downloaded to the target DUT and run different bricked board signatures and version numbers might be displayed, depending on exact configuration details.



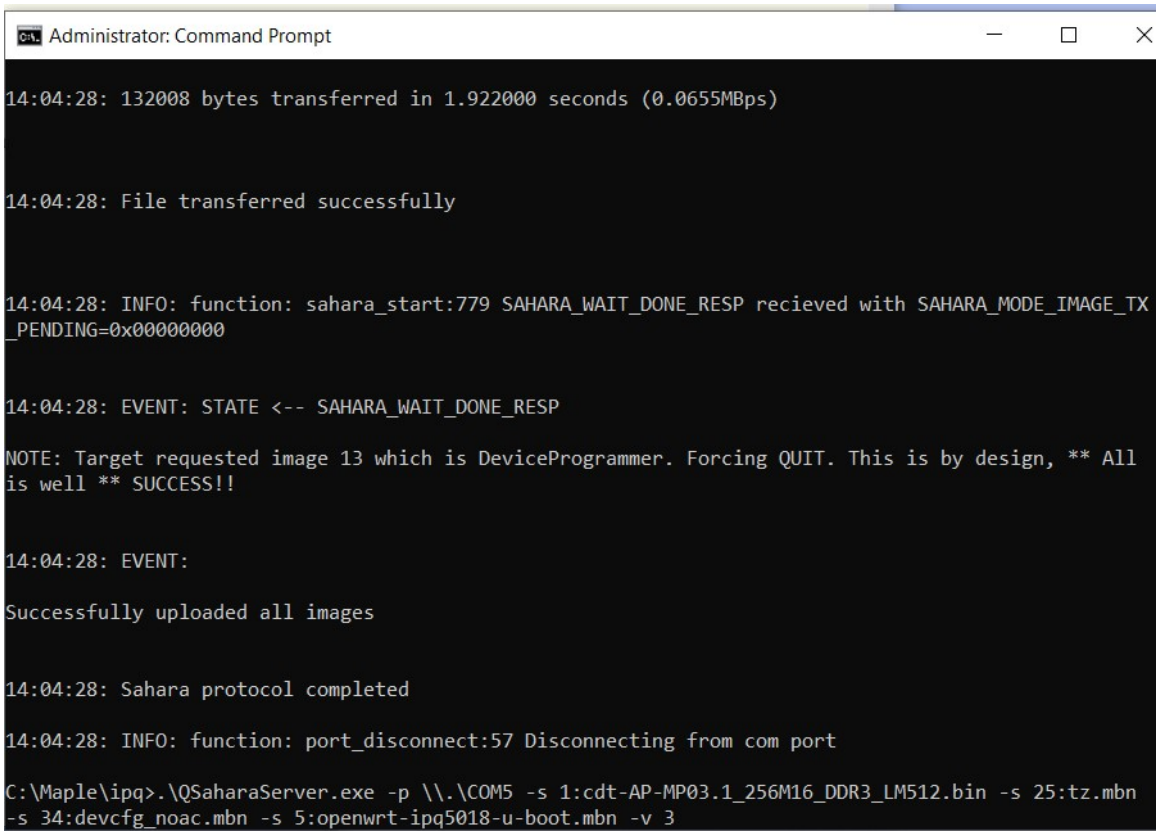
4. After the successful transfer of the image, run this command to transfer other images:

   ```
   QsaharaServer.exe -p \\.\COM<port number> -s 1:<CDT file name> -s
   34:devcfg_noac.mbn -s 25:tz.mbn -s 5:openwrt-ipq5018-u- boot.mbn -v 3
   ```

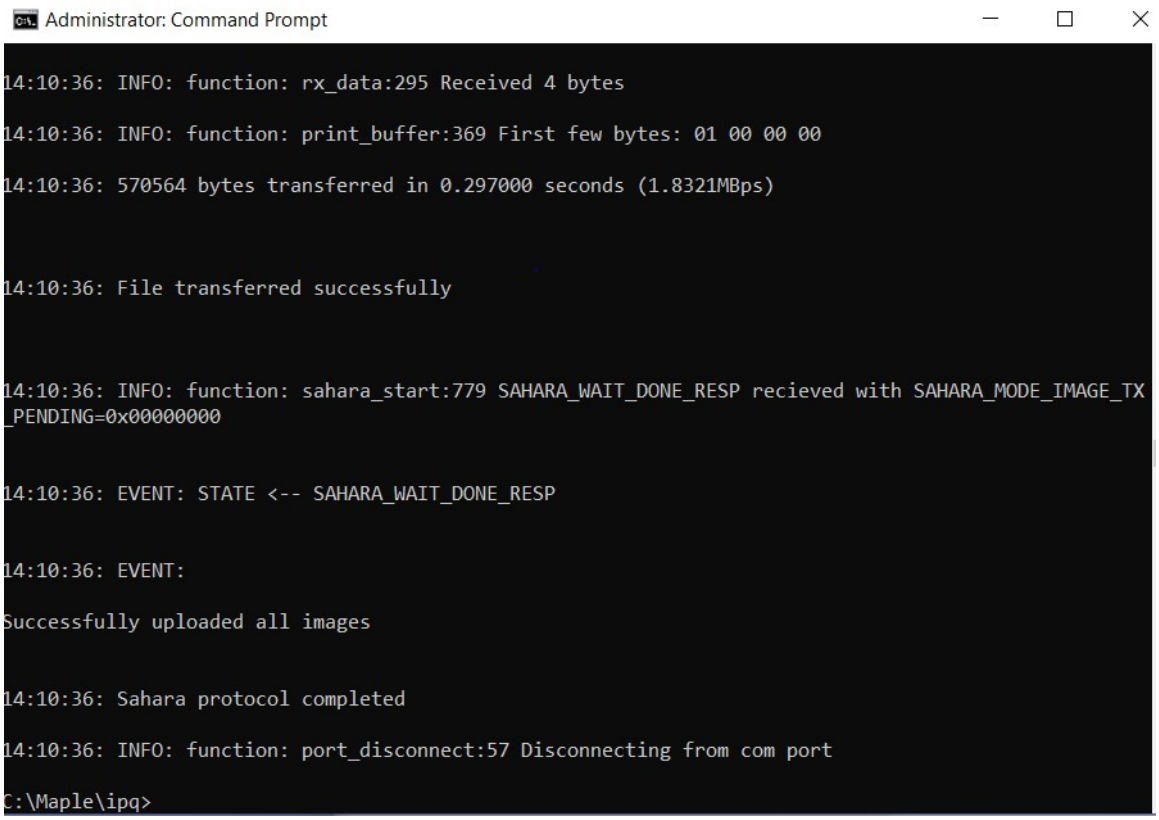
   Where:

   - <*port number*> is the com port number detected on Host PC. See the Device Manager for details.
   - <*CDT file name*> is the correct CDT file for that board.

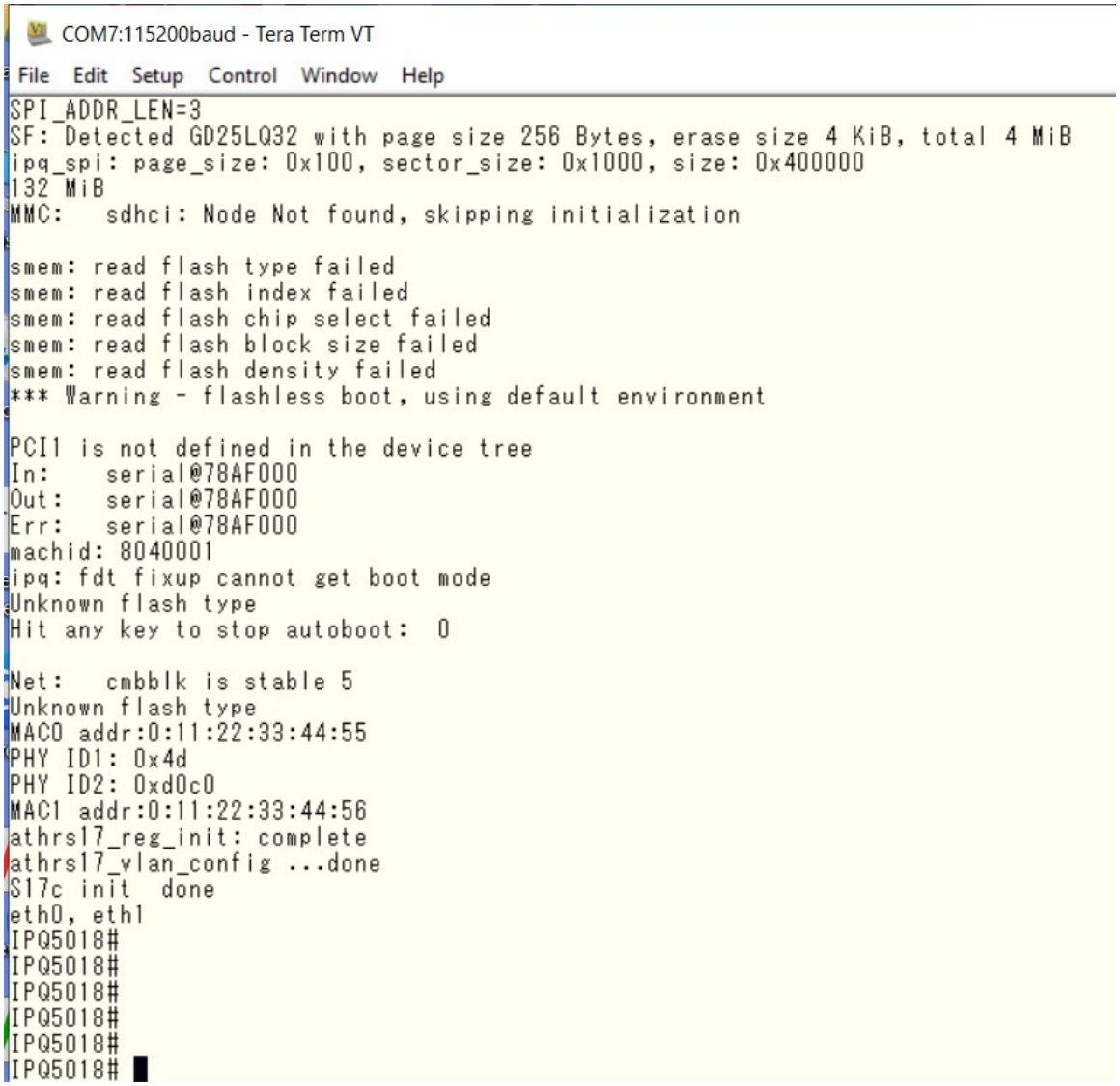
Upon successful transfer of images, confirmation messages are printed to the QSahara window, and bootlog messages are visible in the serial console window (Putty COM5 as shown here).



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



NOTE This procedure immediately runs the initialize clocks and DDR functions, and then loads and runs U-Boot. An error may occur if no kernel image exists in the configured boot flash memory (that is, only SBL/TZ/U-boot partitions were corrupted originally). Stop auto kernel load by pressing any key on the keyboard that is attached to the MP0x serial console (terminal) emulation application.

Select and flash the complete image as described in Section Transfer the image using TFTP or Transfer the image using USB (host mode), and Section Flash the image and perform a hard reboot

11.4 Flash Wi-Fi firmware-only images using TFTP

For custom Wi-Fi firmware changes, flash Wi-Fi firmware images using TFTP in U-Boot. This method is not required if the single image is generated using the Wi-Fi firmware images from the firmware build. It is useful when there is a need to only update the firmware partition with updated firmware images. The firmware images are in **<BuildLocation>/build/ms/bin/FW_IMAGES.**

NOTE The new Wi-Fi FW flash partition is read-only from Linux; and requires an update using U-Boot commands. This method deprecates the previous method of copying PIL images to /lib/firmware/IPQ5018.

11.4.1 NAND boot and NOR + NAND boot

1. Set up the TFTP server.
2. Copy the Wi-Fi firmware binaries from the firmware build location **<BuildLocation>/build/ms/bin/FW_IMAGES** to the TFTP server root folder.
3. In U-Boot, set the board IP address the server IP address:

```
setenv ipaddr <board IP address>
setenv serverip <server IP address>
ping $serverip
```

4. Transfer the image using TFTP:

```
tftpboot 0x44000000 wifi_fw_squashfs.img
```

5. Use smem to get the Wi-Fi firmware UBI volume name.

```
15: rootfs           0x0100ffff          0x80000        0x3e00000
        ubi vol 0 kernel
        ubi vol 1 wifi_fw
        ubi vol 2 bt_fw
        ubi vol 3 ubi_rootfs
        ubi vol 4 rootfs_data
```

   In this example, wifi_fw is the Wi-Fi firmware UBI volume name.

6. Flash the Wi-Fi firmware:

```
flash wifi_fw
```

7. Reboot the device using the reset command.

11.4.2 NOR boot

1. Set up the TFTP server.
2. Copy the Wi-Fi firmware binaries from the firmware build location: **<BuildLocation>/build/ms/bin/FW_IMAGES** to the TFTP server root folder.
3. In U-Boot, set the board IP address and the server IP address:

```
setenv ipaddr <board IP address>
setenv serverip <server IP address>
ping $serverip
```

4. Transfer the image using TFTP:

   ```
   #tftpboot 0x44000000 wifi_fw_squashfs.img
   ```

5. Run the # smem command

   This command gets the Wi-Fi firmware partition name, size, and offset at U-Boot. For example:

   ```
   12: 0:WIFIFW    0x0000ffff    0x01b00000  0x00400000
   ```

6. Flash the Wi-Fi firmware:

   ```
   flash 0:WIFIFW
   ```

7. Reboot the device using the **reset** command.

11.4.3 eMMC boot and NOR + eMMC boot

1. Set up the TFTP server
2. Copy the Wi-Fi firmware binaries from the firmware build location **<BuildLocation>/build/ms/bin/FW_IMAGES** to the TFTP server root folder.
3. In U-Boot, set the board IP address the server IP address:

   ```
   setenv ipaddr <board IP address>
   setenv serverip <server IP address>
   ping $serverip
   ```

4. Transfer the image using TFTP:

   ```
   tftpboot 0x44000000 wifi_fw_squashfs.img
   ```

5. Run the #mmc part command.

   This command gets the Wi-Fi firmware partition name, size, and offset. For example:

   ```
   18     0x00044022      0x00047021      "0:WIFIFW"
   ```

6. Flash the Wi-Fi firmware:

   ```
   flash 0:WIFIFW
   ```

7. Reboot the device using the **reset** command.

11.5 Flash Bluetooth firmware-only images using TFTP

For custom Bluetooth firmware changes, flash Bluetooth firmware images using TFTP in U-Boot. This method is not required if the single image is generated using the Bluetooth firmware images from the firmware build. It is useful when there is a need to update only the firmware partition with updated firmware images. The firmware images are available at, **<BuildLocation>/btfw_proc\out\IPQ5018\bin\FW_IMAGES.**

NOTE The new Bluetooth firmware flash partition is read-only from Linux, and requires an update using U-Boot commands. This method deprecates the previous method of copying PIL images to /lib/firmware/IPQ5018.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

11.5.1 NAND boot and NOR + NAND boot

1. Setup the TFTP server.
2. Copy the Bluetooth firmware binaries from the firmware build location **<BuildLocation>/btfw_proc\out\IPQ5018\bin\FW_IMAGES** to the TFTP server root folder.
3. In U-Boot, set the board IP address the server IP address:

```
setenv ipaddr <board IP address>
setenv serverip <server IP address>
ping $serverip
```

4. Transfer the image using TFTP:

```
tftpboot 0x44000000 bt_fw_patch_squashfs.img
```

5. Use smem to get the Bluetooth firmware UBI volume name.

```
15: rootfs           0x0100ffff          0x80000        0x3e00000
        ubi vol 0 kernel
        ubi vol 1 wifi_fw
        ubi vol 2 bt_fw
        ubi vol 3 ubi_rootfs
        ubi vol 4 rootfs_data
```

   In this example, *bt_fw* is the Bluetooth firmware UBI volume name.

6. Flash the Bluetooth firmware.

```
flash bt_fw
```

7. Reboot the device using the **reset** command.

11.5.2 NOR boot

1. Setup the TFTP server.
2. Copy the Bluetooth firmware binaries from the firmware build location **<BuildLocation>/btfw_proc\out\IPQ5018\bin\FW_IMAGES** to the TFTP server root folder.
3. In U-Boot, set the board IP address the server IP address:

```
setenv ipaddr <board IP address>
setenv serverip <server IP address>
ping $serverip
```

4. Transfer the image using TFTP:

```
#tftpboot 0x44000000 bt_fw_patch_squashfs.img
```

5. Use the smem command to get the Bluetooth firmware partition name, size, and offset at U-Boot. For example:

```
12: 0:BTFW    0x0000ffff    0x01b00000  0x00400000
```

   In this example, *bt_fw* is the Bluetooth firmware UBI volume name.

6. Flash the Bluetooth firmware.

   ```
   flash 0:BTFW
   ```

7. Reboot the device using the **reset** command.

11.5.3 eMMC boot and NOR + eMMC boot

1. Setup the TFTP server.
2. Copy the Bluetooth firmware binaries from the firmware build location **<BuildLocation>/ btfw_proc\out\IPQ5018\bin\FW_IMAGES** to the TFTP server root folder.
3. In U-Boot, set the board IP address the server IP address:

   ```
   setenv ipaddr <board IP address>
   setenv serverip <server IP address>
   ping $serverip
   ```

4. Transfer the image using TFTP:

   ```
   #tftpboot 0x44000000 bt_fw_patch_squashfs.img
   ```

5. Use the #mmc part command to get the Bluetooth firmware partition name, size, and offset at U-Boot. For example:

   ```
   20      0x0004a022      0x0004a221      "0:BTFW"
   ```

   In this example, *bt_fw* is the Bluetooth firmware UBI volume name.

6. Flash the Bluetooth firmware.

   ```
   flash 0:BTFW
   ```

7. Reboot the device using the reset command.

12 SBL UART customization

To customize SBL UART, contact the QTI Customer Engineering (CE) team.